



Universidad Politécnica
de Madrid

**Escuela Técnica Superior de
Ingenieros Informáticos**



Grado en INGENIERÍA INFORMÁTICA

Trabajo Fin de Grado

**Algoritmo de estimación de
distribuciones para resolver el cubo de
Rubik 3x3x3 fuse**

Autor: DANIEL MANSILLA NEBREDA

Tutor: JUAN ANTONIO FERNANDEZ DEL POZO DE SALAMANCA

Madrid, ENERO - 2026

Este Trabajo Fin de Grado se ha depositado en la ETSI Informáticos de la Universidad Politécnica de Madrid para su defensa.

Trabajo Fin de Grado

Grado en INGENIERÍA INFORMÁTICA

Título: Algoritmo de estimación de distribuciones para resolver el cubo de Rubik 3x3x3 fuse

ENERO - 2026

Autor: DANIEL MANSILLA NEBREDÁ

Tutor: JUAN ANTONIO FERNÁNDEZ DEL POZO DE SALAMANCA

DEPARTAMENTO

Escuela Técnica Superior de Ingenieros Informáticos

Universidad Politécnica de Madrid

Índice general

1. Introducción	3
1.1. Motivación del proyecto	3
1.2. Contexto del proyecto	3
1.3. Objetivos	4
1.4. Definición del trabajo	5
1.5. Estructura del Documento	5
2. Trabajo relacionado y Estado del Arte	7
2.1. El cubo de Rubik y sus variantes	7
2.1.1. El cubo de Rubik 3x3x3	7
2.1.2. El cubo <i>bandaged</i> o <i>fuse</i>	8
2.1.3. Modelado del cubo	9
2.1.4. Representación computacional del cubo	10
2.1.5. Función de similitud entre configuraciones	11
2.2. Algoritmos de Estimación de Distribuciones (EDA)	12
2.2.1. Concepto y origen de los EDA	12
2.2.2. Funcionamiento general de un EDA	12
2.2.3. Principales variantes de EDA	13
3. Infraestructura del cubo FUSE	14
3.1. Definición estructural	14
3.2. Sistema de referencia espacial y notación	16
3.3. Codificación de las piezas móviles	17
3.3.1. Esquinas móviles	17
3.3.2. Aristas móviles	18
3.3.3. Bloque rígido e información invariante	19
3.3.4. Identidad de pieza frente a posición	19
3.3.5. Justificación de la codificación elegida	20
3.4. Representación del estado del cubo	20
3.4.1. Conjuntos de posiciones de referencia	21
3.4.2. Permutación de esquinas	21
3.4.3. Orientación de esquinas	22
3.4.4. Permutación de aristas	22
3.4.5. Orientación de aristas	23
3.4.6. Estado resuelto	23
3.5. Modelado de la acción de cada movimiento	24
3.5.1. Función de transición de estados	24
3.5.2. Convención de indexación	25

3.5.3. Acción del movimiento L	26
3.5.4. Acción del movimiento D	26
3.5.5. Acción del movimiento B	27
3.5.6. Movimientos derivados	28
3.5.7. Consistencia del modelo de transición	28
4. Diseño en Implementación del Solver basado en EDAs	29
4.1. Justificación del modelo EDA elegido	29
4.2. Representación del problema como genoma	31
4.2.1. Alfabeto de genes	31
4.2.2. Token STOP y aprendizaje de la longitud efectiva	31
4.2.3. Decodificación y simplificación de cancelaciones	31
4.2.4. Reconstrucción del algoritmo mediante el mejor prefijo	32
4.3. Función de evaluación	32
4.3.1. Primera versión: similitud por coincidencia directa	33
4.3.2. Segunda versión: similitud por longitud de ciclo	33
4.3.3. Versión final: bases de datos de patrones	34
4.3.4. Reward denso por fases	36
4.3.5. Evaluación por máximo de prefijos	37
4.3.6. Penalización por longitud	38
4.4. Inicialización informada del modelo probabilístico	38
4.4.1. Distribución uniforme y sus limitaciones	39
4.4.2. Penalización de cara repetida en posiciones contiguas	39
4.4.3. Sesgo por el inverso del scramble	40
4.4.4. Generación coherente de la primera población	41
4.5. Estrategia de exploración con reinicios	41
4.5.1. Longitud creciente del genoma	41
4.5.2. Reinicios cálidos: sesgo hacia el mejor individuo conocido	42
4.5.3. Mezcla de frecuencias por estancamiento dentro de un reinicio	42
4.5.4. Registro de élites	43
4.6. Fortalezas del diseño	43
4.7. Limitaciones del diseño	44
5. Resultados y conclusiones	45
5.1. Resultados	45
5.1.1. Metodología experimental	45
5.1.2. Validación funcional del cubo	46
5.1.3. Resultados del solver	47
5.1.4. Verificación independiente de las soluciones	49
5.1.5. Discusión de resultados	49
5.2. Conclusiones personales	50
5.3. Trabajo futuro	51
6. Impacto del trabajo	53
6.1. Impacto general	53
6.2. Objetivos de Desarrollo Sostenible	54
Bibliografía	56
A. Anexo	58

Índice de Figuras

3.1. Comparación entre el cubo de Rubik clásico y el cubo <i>fuse</i>	15
4.1. Flujo general del solver basado en $UMDA_{cat}$	30
4.2. Comparación de las funciones de evaluación según la longitud del scramble.	36
4.3. Construcción del coste utilizado por $UMDA_{cat}$ a partir de la señal basada en PDB, el reward denso, la evaluación por mejor prefijo y la penalización por longitud.	38
5.1. Tasa de resolución del solver según la longitud del scramble.	48

Índice de Tablas

5.1. Resultados del solver por profundidad de scramble.	47
---	----

Índice de Listings

Resumen

Este Trabajo Fin de Grado trata sobre la resolución de una variante del cubo de Rubik 3x3x3 conocida como cubo *fuse* o *bandaged cube*. En esta variante, algunas piezas del cubo están unidas entre sí, lo que limita los movimientos que se pueden realizar y hace que el problema sea distinto al del cubo de Rubik convencional.

El objetivo principal del trabajo es diseñar e implementar un sistema capaz de resolver, o al menos acercarse a la solución, de un cubo *fuse* mediante un Algoritmo de Estimación de Distribuciones (EDA). Para ello, primero se desarrolla una representación del cubo que permite modelar sus piezas, sus orientaciones y los movimientos válidos. Después, se implementa un solver basado en EDA, donde cada individuo representa una posible secuencia de movimientos.

Durante el desarrollo se prueban distintas formas de evaluar la cercanía de un estado al cubo resuelto. Entre ellas se incluyen una función de similitud directa, una función basada en ciclos de permutación y una medida basada en bases de datos de patrones. También se añaden mecanismos como el token *STOP*, la simplificación de secuencias, la evaluación por mejor prefijo y una estrategia de reinicios con longitudes de genoma crecientes.

Por último, se realizan pruebas para comprobar que la implementación del cubo funciona correctamente y para analizar el comportamiento del solver con scrambles de distintas longitudes. Los resultados permiten observar hasta qué punto el algoritmo es capaz de resolver el cubo, cómo cambia su rendimiento cuando aumenta la dificultad y cuáles son sus principales limitaciones. El objetivo del trabajo no es obtener un solver óptimo, sino estudiar si un EDA puede aplicarse a este tipo de cubo con restricciones.

Palabras Clave: Cubo de Rubik, Cubo Fuse, Bandaged Cube, Algoritmos de Estimación de Distribuciones, EDA, Optimización Combinatoria, Inteligencia Artificial, Solver.

Abstract

This Bachelor's Thesis focuses on solving a variant of the 3x3x3 Rubik's Cube known as the *fuse cube* or *bandaged cube*. In this variant, some pieces of the cube are joined together, which limits the movements that can be performed and makes the problem different from the standard Rubik's Cube.

The main goal of this project is to design and implement a system capable of solving, or at least approaching the solution of, a *fuse cube* using an Estimation of Distribution Algorithm (EDA). To do this, a representation of the cube is first developed in order to model its pieces, orientations and valid movements. Then, an EDA-based solver is implemented, where each individual represents a possible sequence of movements.

During the development, different ways of evaluating how close a state is to the solved cube are tested. These include a direct similarity function, a function based on permutation cycles and a measure based on pattern databases. Other mechanisms are also added, such as a *STOP* token, sequence simplification, best-prefix evaluation and a restart strategy with increasing genome lengths.

Finally, tests are carried out to check that the cube implementation works correctly and to analyse the behaviour of the solver with scrambles of different lengths. The results show how far the algorithm is able to solve the cube, how its performance changes as the difficulty increases and what its main limitations are. The aim of this work is not to obtain an optimal solver, but to study whether an EDA can be applied to this type of constrained cube.

Keywords: Rubik's Cube, Fuse Cube, Bandaged Cube, Estimation of Distribution Algorithms, EDA, Combinatorial Optimization, Artificial Intelligence, Solver.

Capítulo 1

Introducción

En esta sección se abordará la motivación del proyecto, así como su contexto, una lista de objetivos que se marcarán para este TFG y el resumen de este.

1.1. Motivación del proyecto

El objetivo que se quiere conseguir es principalmente llegar o quedarse cerca de una solución y poniendo el foco más que en la eficiencia del algoritmo, en cómo va aprendiendo este y que datos le vamos pasando para que se vaya acercando a una solución.

También se quiere abordar un nuevo campo dentro de los EDAs que es aplicarlos en un entorno combinatorio con restricciones que no se había tratado hasta ahora así que aportaríamos valor al desarrollo y al conocimiento de los EDA.

Es otra motivación resolver un cubo que no es el habitual (el 3x3x3) de una forma poco común que nunca se ha aplicado en estos cubos así se dará más visibilidad a otro tipo de rompecabezas más desconocidos como es el fuse cube a diferencia del cubo de Rubik convencional, que tiene muchos estudios de maneras de resolución de este.

1.2. Contexto del proyecto

Desde su creación en 1974, el cubo de Rubik ha sido uno de los problemas o rompecabezas más estudiados a nivel computacional. Ya desde el inicio empezaron a aparecer algoritmos de resolución de este así como aplicar algoritmos ya existentes para obtener el mismo resultado. Los métodos que se han ido utilizando van desde el uso de la inteligencia artificial, algoritmos de optimización o algoritmos de búsqueda. Richard Earl Korf en 1997 redactó y codificó el primer programa capaz de resolver de manera óptima el cubo de Rubik; y el algoritmo que implementó, consistía en algoritmos de búsqueda informada y el uso de base de datos de patrones y esto sentó las bases de los métodos de resolución óptimos del cubo 3x3x3 [1].

También, respecto al cubo que tenemos en frente, en el que alguna de las piezas están unidas, surgió como una variante del cubo principal y se le denomina a este tipo de cubos como bandaged cubes o fuse cubes por la comunidad de fans del cubo de Rubik. Este tipo de cubos cuentan con muchas restricciones de movimientos y

hacen que su resolución sea distinta al del cubo normal, elevando su complejidad si lo queremos resolver computacionalmente. Sin embargo no hay muchos trabajos por no decir ninguno que traten sobre la resolución algorítmica de estos cubos.

Últimamente los algoritmos evolutivos se han convertido en la alternativa más eficaz cuando el espacio o la muestra es compleja y poco estructurada. Dentro de esta rama se encuentran los algoritmos de estimación de distribuciones EDAs que destacan por utilizar un modelo probabilístico que se va a formar a base de las mejores soluciones obtenidas en cada generación. Estos pueden detectar dependencias entre variables, lo que los hace útiles para la resolución de este tipo de problemas en los que las relaciones estructurales no son relevantes como pasa con las piezas fusionadas del cubo. Como se puede ver en el trabajo de Pedro Larrañaga y Concha Bielza [2], los EDAs se han implementado de manera exitosa en problemas de combinatoria.

Durante los últimos años, debido al rápido avance de la IA han surgido nuevas maneras de resolución del cubo como puede ser la planteada en 2019 por Agostinelli [3] utilizando deep reinforcement learning así como búsqueda heurística. Pero hay que tener en cuenta que todos o casi todos de los trabajos que salen sobre métodos de resolución del cubo de Rubik utilizan el cubo 3x3x3 y no indagan en aquellos que como el bandaged tienen restricciones de movimiento.

En este contexto que he planteado la propuesta de TFG que se está exponiendo se introduce una nueva forma de resolución del cubo de Rubik y se cuenta que este tiene restricciones por lo que estaría abordando una solución que no se había dado hasta ahora en el campo de la inteligencia artificial aplicada a la resolución de problemas de este estilo como la resolución de este rompecabezas.

1.3. Objetivos

El objetivo principal que se tiene a la hora de hacer este TFG es diseñar un sistema que sea capaz de resolver o si no resolver, quedarse cerca de lo que es la resolución de un cubo fuse 3x3x3 y el algoritmo que se utilizará es un EDA y no se buscará la optimalidad.

- Modelización y representación del cubo en un entorno digital:
Se va a diseñar una representación del cubo fuse que sea eficiente y sencilla a la hora de aplicar el algoritmo y se tendrán en cuenta las restricciones de movimientos así como la estructura de este.
- Implementación y desarrollo de un algoritmo EDA:
Desarrollo de un algoritmo EDA para la resolución del cubo fuse es decir que trabajará en un entorno con restricciones, este EDA será capaz de aprender y modelar modelos de probabilidad a partir de las soluciones que se vayan aproximando a la solución final del cubo.
- Combinar conocimiento del dominio del cubo con el EDA:
Establecer funciones u objetivos basados en trucos o en reglas de la resolución del cubo para ir guiando al EDA a la búsqueda de la solución.
- Extraer conocimiento del algoritmo de solución en términos de estrategias de mejora.:

Introducción

Una vez realizadas las pruebas se obtendrán los resultados de estas y se realizará un estudio de la posible mejora del EDA en base a los resultados obtenidos.

- Analizar el rendimiento del algoritmo :
Una vez realizadas todas las pruebas y obtenido los resultados, se expondrá las limitaciones y fortalezas del modelo implementado así como proponer mejoras o aspectos que pudieran mejorar a futuro de lo implementado en este EDA.

1.4. Definición del trabajo

Este trabajo trata principalmente de la resolución de un cubo de Rubik no convencional como es el 3x3x3 fuse mediante el uso de algoritmo de estimación de distribuciones EDA.

El cubo fuse (lo llamaré de este de ahora en adelante) consiste en un cubo de rubik convencional 3x3x3 pero con la distinción alguna de sus piezas están fusionadas entre sí formando una sola pieza y esto restringe algunos movimientos. La mejor descripción que podríamos dar de este cubo modificado es que tres de sus caras tienen 4 piezas unidas creando una sola pieza, es decir estas 3 caras solo tendrán 6 piezas cada una siendo 5 de ellas piezas normales (2 aristas y 5 esquinas) y una grande unida (formada por el centro dos aristas y la esquina que colinda entre las dos aristas formando un gran cuadrado entre las 4).

La unión de estas piezas hace que el cubo pierda grados de libertad ya que restringe bastante los movimientos del cubo llevando a que su resolución sea más compleja que la de un cubo de Rubik 3x3x3 normal.

La resolución de este cubo se va a llevar a cabo mediante el uso de un algoritmo de estimación de distribuciones (EDA). Este algoritmo que se usará pertenece a la los algoritmos evolutivos, que son los que utilizan procesos de selección y mejora a raíz de una serie de muestras que se le suministran. Otra característica es que este algoritmo no utiliza métodos ni de mutación, ni de cruce; en cambio, lo que hacen es generar una nueva población de individuos a base o a raíz de una distribución de probabilidad, la cual se estima teniendo en cuenta el conjunto de población generado anteriormente en iteraciones previas y poco a poco se va llegando a una solución óptima a medida que se va ajustando o ejecutando el algoritmo.

Lo que se quiere o se intenta conseguir con este proyecto es llegar (o acercarse) a una solución de un cubo de Rubik no convencional, que cuenta con restricciones a raíz de un EDA, así como se pone el foco sobre todo en esto último que es la implementación de un EDA para este caso ya que no se suele utilizar este tipo de algoritmos para la resolución del cubo de Rubik y el diseño del cubo presentaría también un reto adicional debido a las restricciones y a como representar o implementar el cubo.

1.5. Estructura del Documento

El documento se organiza en cinco capítulos principales.

En el Capítulo 1 se presenta la introducción del trabajo. En él se explica la motivación

del proyecto, el contexto en el que se sitúa, los objetivos que se pretenden alcanzar y la definición general del problema que se va a tratar.

En el Capítulo 2 se desarrolla el trabajo relacionado y el estado del arte. Primero se introduce el cubo de Rubik 3x3x3 y algunas de sus variantes, prestando especial atención al cubo *fuse* o *bandaged cube*. Después se explican distintas formas de representar computacionalmente el cubo y se presentan los Algoritmos de Estimación de Distribuciones, que son la base del solver implementado.

En los Capítulos 3 y 4 se describe el desarrollo del proyecto. En primer lugar, se detalla la infraestructura del cubo *fuse*, incluyendo la representación de esquinas, aristas, orientaciones, movimientos permitidos y estado resuelto. A continuación, se explica el diseño del algoritmo EDA, la representación del genoma, las funciones de evaluación utilizadas y las mejoras incorporadas durante el desarrollo del solver.

En el Capítulo 5 se presentan las pruebas realizadas y los resultados obtenidos. Primero se validan los componentes principales del cubo, como los movimientos, los inversos y el estado resuelto. Después se analiza el comportamiento del solver con scrambles de distintas longitudes, estudiando la tasa de éxito, la similitud alcanzada, el tiempo de ejecución y las limitaciones observadas. Finalmente, se recogen las conclusiones del trabajo y posibles líneas de mejora futuras.

En el Capítulo 6 se analiza el impacto del trabajo. En este capítulo se comenta el impacto general del proyecto desde el punto de vista académico y técnico, así como su relación con los Objetivos de Desarrollo Sostenible.

Capítulo 2

Trabajo relacionado y Estado del Arte

En este capítulo se presenta el contexto teórico del trabajo. Primero se introduce el cubo de Rubik 3x3x3 y algunas de sus variantes, con especial atención al cubo *fuse* o *bandaged cube*. Después se explican distintas formas de representar computacionalmente el cubo y, finalmente, se presentan los Algoritmos de Estimación de Distribuciones, que son la base del solver desarrollado.

2.1. El cubo de Rubik y sus variantes

2.1.1. El cubo de Rubik 3x3x3

El cubo de Rubik 3x3x3 fue inventado en el año 1974 por el profesor húngaro Ernő Rubik, con el objetivo de trabajar conceptos relacionados con la geometría tridimensional. Después de desarrollar varios prototipos, consiguió crear un mecanismo que permitía mover las piezas sin que el cubo se desmontase. Al mezclarlo, observó que el objeto planteaba un problema de resolución mucho más complejo de lo esperado [4].

Posteriormente, en 1975, Ernő Rubik solicitó la patente del mecanismo, que inicialmente fue conocido como *Magic Cube*. Más adelante, el cubo comenzó a producirse y distribuirse internacionalmente hasta adoptar el nombre de *Rubik's Cube* [5].

A finales de los años 70 e inicios de los años 80, el cubo empezó a venderse de forma masiva en distintos países. Su popularidad creció rápidamente y se convirtió en uno de los rompecabezas más conocidos de la época, llegando a vender decenas de millones de unidades [6].

Además de su impacto cultural, el cubo de Rubik también despertó interés dentro de la comunidad científica y matemática. No solo se trata de un rompecabezas, sino

también de un problema relacionado con la combinatoria, la teoría de grupos y los algoritmos de búsqueda. El cubo clásico tiene aproximadamente 4.3×10^{19} configuraciones posibles, lo que lo convierte en un problema adecuado para estudiar métodos de optimización y resolución automática [7].

2.1.2. El cubo *bandaged* o *fuse*

El cubo de Rubik *fuse* o *bandaged* es una variante del cubo clásico 3x3x3 en la que algunas piezas están unidas entre sí, formando bloques rígidos. Esta unión impide realizar algunos movimientos que sí serían válidos en el cubo convencional, por lo que el espacio de movimientos queda restringido.

En el caso concreto tratado en este TFG, se considera un cubo *fuse* con un bloque rígido situado en la región superior-derecha-frontal del cubo. Este bloque integra la esquina correspondiente a esa región, tres aristas adyacentes y tres centros. Como consecuencia, dichas piezas no se tratan como elementos móviles independientes dentro del modelo. Bajo la convención utilizada en este trabajo, el cubo queda representado mediante 7 esquinas móviles, 9 aristas móviles y un bloque rígido fijo.

Desde el punto de vista combinatorio, cada una de las 7 esquinas móviles puede ocupar una de las 7 posiciones disponibles y presentar 3 orientaciones posibles. Del mismo modo, cada una de las 9 aristas móviles puede ocupar una de las 9 posiciones disponibles y presentar 2 orientaciones posibles. Sin considerar restricciones adicionales de alcanzabilidad, el número bruto de configuraciones posibles vendría dado por:

$$|S|_{\text{bruto}} = 7! \cdot 3^7 \cdot 9! \cdot 2^9 = 2\,047\,916\,821\,708\,800 \approx 2.05 \times 10^{15}.$$

Sin embargo, no todas estas configuraciones tienen por qué ser alcanzables mediante movimientos legales del cubo. Al igual que ocurre en el cubo de Rubik clásico, existen restricciones asociadas a la orientación de las esquinas, la orientación de las aristas y la paridad de las permutaciones. En la representación por *cubies*, el cubo no se describe únicamente mediante permutaciones, sino también mediante orientaciones de esquinas y aristas [8]. Aplicando restricciones análogas a las del cubo clásico, se obtiene la siguiente cota superior:

$$|S| \leq \frac{7! \cdot 3^7 \cdot 9! \cdot 2^9}{12} = 170\,659\,735\,142\,400 \approx 1.71 \times 10^{14}.$$

Este valor debe interpretarse como una cota superior del espacio de estados considerado, no como el tamaño exacto del conjunto de estados alcanzables. Debido a las restricciones propias del cubo *fuse* y al uso de un conjunto reducido de movimientos legales, el espacio realmente alcanzable podría ser menor. Aun así, esta estimación permite comparar el tamaño del problema con el cubo de Rubik clásico, cuyo espacio de estados es del orden de 4.3×10^{19} configuraciones [1].

Esta modificación hace que el cubo tenga un comportamiento distinto al del cubo de Rubik clásico. Aunque el número de piezas móviles se reduce, el problema no necesariamente se vuelve más sencillo, ya que algunos caminos de resolución dejan de ser posibles al estar prohibidos ciertos movimientos. Desde un punto de vista computacional, esto obliga a adaptar tanto la representación del estado como el conjunto de operadores que puede aplicar el solver.

En un cubo clásico, el espacio de estados puede estudiarse mediante el grupo generado por los movimientos de sus caras. En el caso del cubo *fuse*, las restricciones introducidas por el bloque rígido modifican el grafo de estados alcanzables. Por ello, aunque el espacio de búsqueda puede ser menor, también puede estar más restringido y presentar caminos de resolución menos directos que en el cubo convencional.

2.1.3. Modelado del cubo

Representación general

Para representar el cubo se parte de una descripción de sus centros, esquinas y aristas. En las figuras correspondientes se muestran las piezas que forman parte del bloque rígido y las piezas que se consideran móviles dentro del modelo.

Como se observa en la representación del cubo, existen 6 centros, de los cuales 3 forman parte del bloque rígido considerado en este trabajo. En cuanto a las esquinas, el modelo representa 7 esquinas móviles, ya que la esquina restante queda integrada en el bloque fijo. Cada esquina puede presentar 3 orientaciones posibles. Por último, el modelo representa 9 aristas móviles, y cada una de ellas puede tener 2 orientaciones posibles.

Representación de los giros

Para describir los movimientos del cubo se utiliza la notación habitual basada en las caras: *Up*, *Down*, *Front*, *Back*, *Left* y *Right*. Cada movimiento indica qué cara del cubo se gira respecto a una orientación inicial fijada. En este trabajo se parte de una orientación concreta del cubo, en la que el bloque rígido queda situado en la zona superior-derecha-frontal.

Los giros pueden ser de 90 grados, 180 grados o -90 grados. Sin embargo, debido a las restricciones del cubo *fuse*, no todas las caras pueden girarse. En el modelo considerado solo se permiten giros sobre las caras L, D y B, junto con sus variantes dobles e inversas. Por tanto, el conjunto de movimientos utilizado por el solver es:

$$\{L, L2, L', D, D2, D', B, B2, B'\}.$$

2.1.4. Representación computacional del cubo

Representación *facelets*

Una de las formas habituales de representar el cubo es mediante *facelets*. Esta representación modela cada una de las pegatinas visibles del cubo. Cada pegatina se codifica mediante un valor asociado a su color, formando una estructura equivalente a una matriz de $6 \times 3 \times 3$ en el caso del cubo convencional [9].

Representación basada en *cubies*

Otra representación habitual es la basada en *cubies*. En lugar de modelar únicamente los colores visibles, esta representación describe el estado interno del cubo mediante la permutación y orientación de sus piezas. De este modo, el estado se separa en permutaciones de esquinas, orientaciones de esquinas, permutaciones de aristas y orientaciones de aristas.

Este modelo es frecuente en solvers del cubo de Rubik porque permite representar el estado de forma compacta y aplicar movimientos como transformaciones sobre vectores de permutación y orientación. También se ha utilizado en trabajos clásicos de resolución óptima del cubo, como el de Korf en 1997 [1].

Representación algebraica

Desde un punto de vista algebraico, el cubo de Rubik puede representarse mediante el grupo generado por sus movimientos básicos. En el cubo clásico, los movimientos de las caras *Up*, *Down*, *Left*, *Right*, *Front* y *Back* actúan como permutaciones sobre las piezas del cubo. Por ello, el conjunto de configuraciones alcanzables puede estudiarse como un grupo de permutaciones, normalmente expresado como un subgrupo de un grupo simétrico [10].

También existen representaciones mediante vectores o matrices que codifican la posición de las piezas y que pueden utilizarse como entrada para algoritmos de inteligencia artificial, como ocurre en trabajos basados en aprendizaje por refuerzo profundo [3].

Representación para el cubo *fuse*

Para representar el cubo *fuse* de este trabajo se utiliza una representación basada en *cubies*, pero adaptada a las restricciones del bloque rígido. Mientras que el cubo de Rubik clásico tiene 8 esquinas y 12 aristas, el modelo empleado en este trabajo solo representa 7 esquinas móviles y 9 aristas móviles. Además, el conjunto de movimientos disponibles queda reducido a los giros legales descritos en la sección anterior.

2.1.5. Función de similitud entre configuraciones

Dada la configuración del cubo *fuse*, se consideran 7 esquinas móviles, 9 aristas móviles y un bloque rígido que permanece invariante. La similitud entre dos configuraciones se calcula comparando únicamente las esquinas y aristas móviles. Se denota por A el estado actual del cubo y por B el estado objetivo, correspondiente al cubo resuelto. Para cada esquina i del conjunto $|C| = 7$ y cada arista j del conjunto $|D| = 9$, se definen los siguientes indicadores:

$$\text{pos}(A, B) = \begin{cases} 1 & \text{si la misma pieza ocupa la misma posición en } A \text{ y } B \\ 0 & \text{en otro caso} \end{cases}$$

$$\text{ori}(A, B) = \begin{cases} 1 & \text{si la orientación de la pieza en } A \text{ coincide con la de } B \\ 0 & \text{en otro caso} \end{cases}$$

En el caso de las esquinas se consideran tres orientaciones posibles, mientras que en el caso de las aristas se consideran dos orientaciones posibles [9].

A partir de estos indicadores se definen las siguientes medidas parciales:

$$\begin{aligned} \text{posC}(A, B) &= \frac{1}{7} \sum_{i \in C} \text{pos}_i(A, B) \\ \text{oriC}(A, B) &= \frac{1}{7} \sum_{i \in C} (\text{pos}_i(A, B) \cdot \text{ori}_i(A, B)) \\ \text{posD}(A, B) &= \frac{1}{9} \sum_{j \in D} \text{pos}_j(A, B) \\ \text{oriD}(A, B) &= \frac{1}{9} \sum_{j \in D} (\text{pos}_j(A, B) \cdot \text{ori}_j(A, B)) \end{aligned}$$

La orientación solo se tiene en cuenta cuando la pieza se encuentra en su posición correcta. Con ello se evita sobreestimar la similitud en estados donde una pieza pueda tener la orientación adecuada, pero estar situada en una posición incorrecta.

La similitud total se obtiene combinando estas cuatro medidas:

$$\boxed{\text{Sim}(A, B) = \alpha_1 \text{posC}(A, B) + \alpha_2 \text{oriC}(A, B) + \alpha_3 \text{posD}(A, B) + \alpha_4 \text{oriD}(A, B)}$$

sujeta a

$$\alpha_k \geq 0, \quad \sum_{k=1}^4 \alpha_k = 1,$$

de modo que $\text{Sim}(A, B) \in [0, 1]$. Multiplicando este valor por 100 se obtiene el porcentaje de similitud.

2.2. Algoritmos de Estimación de Distribuciones (EDA)

En este trabajo se asignan los siguientes valores a los pesos:

$$(\alpha_1, \alpha_2, \alpha_3, \alpha_4) = (0.35, 0.15, 0.40, 0.10).$$

Esta elección da más peso a la posición que a la orientación, ya que la posición de las piezas tiene mayor impacto en la resolución. Además, se da un peso ligeramente mayor a las aristas, al haber más aristas móviles que esquinas móviles en el modelo utilizado.

2.2. Algoritmos de Estimación de Distribuciones (EDA)

2.2.1. Concepto y origen de los EDA

Los Algoritmos de Estimación de Distribuciones, o EDAs, pertenecen a la familia de los algoritmos evolutivos. A diferencia de los algoritmos genéticos clásicos, que suelen utilizar operadores como cruce y mutación, los EDAs generan nuevas soluciones a partir de un modelo probabilístico aprendido de las mejores soluciones encontradas hasta el momento.

El funcionamiento general consiste en evaluar una población de individuos, seleccionar los mejores, estimar una distribución de probabilidad a partir de ellos y muestrear esa distribución para generar una nueva población. Dependiendo del tipo de EDA, el modelo probabilístico puede asumir independencia entre variables o intentar representar dependencias entre ellas.

Los EDAs comenzaron a desarrollarse especialmente a finales de los años 90 e inicios de los 2000. Una referencia importante es el trabajo de Larrañaga y Lozano [11], donde se presentan distintas variantes de EDAs y se clasifican según la complejidad del modelo probabilístico utilizado. Posteriormente, Hauschild y Pelikan [12] realizaron una revisión amplia sobre estos algoritmos y sus aplicaciones.

2.2.2. Funcionamiento general de un EDA

Los EDAs siguen un ciclo iterativo que combina optimización y aprendizaje. En primer lugar, se genera una población inicial, que puede ser aleatoria o estar construida mediante algún criterio previo. A continuación, se evalúa cada individuo mediante una función de fitness o función objetivo. Después, se selecciona un subconjunto de los mejores individuos y se ajusta un modelo probabilístico a partir de ellos.

Una vez estimado el modelo, se generan nuevos individuos mediante muestreo de la distribución obtenida. Esta nueva población sustituye total o parcialmente a la anterior, y el proceso se repite hasta alcanzar un criterio de parada. Este criterio puede ser un número máximo de generaciones, un número máximo de evaluaciones, la ausencia de mejora durante varias iteraciones o la obtención de una solución válida.

2.2.3. Principales variantes de EDA

Los EDAs se pueden clasificar según el tipo y la complejidad del modelo probabilístico utilizado. Siguiendo esta clasificación, se pueden distinguir modelos univariados, modelos bivariados y modelos multivariados [13].

Modelos sin dependencias o univariados

Los modelos univariados asumen que no existen dependencias entre las variables del individuo. Por ello, calculan una distribución marginal para cada variable y generan nuevos individuos muestreando cada posición de forma independiente. La distribución conjunta se obtiene como producto de las distribuciones marginales individuales.

Este tipo de modelos es más sencillo y tiene menor coste computacional, pero también presenta una limitación importante: no puede capturar relaciones entre distintas posiciones del genoma. En el contexto de un solver basado en secuencias de movimientos, esto puede dificultar el aprendizaje de patrones formados por varios movimientos consecutivos.

Modelos con dependencias bivariadas

Los modelos bivariados permiten representar dependencias entre pares de variables. En estos modelos, la probabilidad de una variable puede depender como máximo de otra variable, normalmente representada como su padre dentro de una estructura de árbol. De esta forma, el modelo puede capturar relaciones simples entre variables sin llegar al coste de los modelos multivariados completos.

Modelos con dependencias múltiples o multivariados

Los modelos multivariados utilizan estructuras más complejas para representar dependencias entre varias variables. Entre ellos se encuentran los modelos basados en redes bayesianas y redes de Markov.

Las redes bayesianas se representan mediante grafos dirigidos acíclicos, en los que los nodos representan variables y las aristas indican dependencias probabilísticas. Por su parte, las redes de Markov utilizan grafos no dirigidos para representar relaciones de dependencia entre variables.

Dentro de los modelos multivariados también existen enfoques basados en productos marginales, donde las variables se agrupan en subconjuntos o *clusters*. Estos modelos buscan un equilibrio entre la capacidad de representar dependencias y el coste computacional asociado al aprendizaje del modelo.

Capítulo 3

Infraestructura del cubo FUSE

En este capítulo se presenta la infraestructura computacional del cubo *fuse*: su definición estructural, el sistema de referencia espacial y la notación empleada, la codificación de las piezas móviles, la representación del estado y el modelado de la acción de cada movimiento. Esta infraestructura constituye la base sobre la que se construirá el solver basado en EDA descrito en el capítulo siguiente.

3.1. Definición estructural

En un cubo de Rubik clásico $3 \times 3 \times 3$, las piezas móviles pueden describirse mediante esquinas, aristas y centros, sobre los que actúan los giros elementales de las seis caras. Sin embargo, el cubo *fuse* utilizado en este trabajo introduce una modificación estructural: varias piezas dejan de comportarse como unidades independientes y pasan a formar un único bloque rígido. Esta característica convierte el problema en una variante *bandaged*, es decir, una versión parcialmente bloqueada del cubo convencional en la que algunos movimientos del cubo clásico dejan de formar parte del conjunto de operadores permitidos.

En este caso, la modificación consiste en la presencia de una pieza rígida de tamaño $2 \times 2 \times 2$, situada en la esquina formada por las caras blanca, azul y roja. Esta pieza engloba la información visible correspondiente a tres centros, tres aristas adyacentes y la esquina común a dichas aristas. Como consecuencia, la estructura resultante deja de ser homogénea: tres caras del cubo quedan condicionadas por la presencia de este bloque, mientras que el modelo mantiene como operadores legales los giros de las caras *L*, *D* y *B*. Esta asimetría es la propiedad que distingue al cubo *fuse* del cubo $3 \times 3 \times 3$ convencional y obliga a redefinir tanto el espacio de estados como el conjunto de movimientos permitidos.

Esta modificación altera también el número convencional de aristas y esquinas móviles que se codifican para su resolución. Mientras que el cubo clásico dispone de 8 esquinas móviles y 12 aristas móviles, en el cubo *fuse* considerado en este trabajo se representan 7 esquinas y 9 aristas móviles, manteniéndose además una pieza $2 \times 2 \times 2$ inmutable. Aunque el número de piezas libres es menor, la restricción geométrica introducida por el bloque rígido elimina movimientos que sí están disponibles en el cubo estándar.



Cubo de Rubik clásico $3 \times 3 \times 3$.

Cubo *fuse* con el bloque rígido
 $2 \times 2 \times 2$.

Figura 3.1: Comparación entre el cubo de Rubik clásico y el cubo *fuse*.

Este punto es importante desde el punto de vista computacional, ya que en muchos problemas combinatorios una reducción del espacio de búsqueda suele asociarse a una menor dificultad de resolución. Sin embargo, en el cubo *fuse* no ocurre necesariamente así. La presencia de restricciones mecánicas reduce el conjunto de movimientos disponibles y puede eliminar caminos que en el cubo clásico sí serían válidos. Por tanto, la complejidad del problema no depende únicamente del número bruto de configuraciones posibles, sino también de la conectividad entre dichas configuraciones.

Para fijar el marco espacial del problema, se adopta una orientación de referencia concreta del cubo. La cara frontal se identifica con el color blanco, la cara superior con azul, la cara derecha con rojo, la cara izquierda con naranja, la cara inferior con verde y la cara trasera con amarillo. Bajo esta convención, el bloque $2 \times 2 \times 2$ queda situado en la región superior-derecha-frontal del cubo. Esta elección proporciona una base estable para nombrar tanto las caras como las posiciones de las piezas y permite describir de forma consistente los movimientos admisibles y su efecto sobre el estado interno del cubo.

A partir de esta referencia espacial, el cubo deja de admitir el conjunto completo de giros característico del $3 \times 3 \times 3$. En el modelo concreto de este cubo, únicamente permanecen disponibles los giros asociados a las caras izquierda (L), inferior (D) y trasera (B), junto con sus variantes de cuarto de vuelta, media vuelta y giro inverso. Dicho de otro modo, el espacio de movimientos se restringe a:

$$\{L, L^2, L', D, D^2, D', B, B^2, B'\}.$$

Esta notación sigue la convención estándar utilizada en el cubo de Rubik. Una letra representa un giro de 90° de la cara correspondiente, el superíndice 2 indica un giro de 180° , y el apóstrofo indica un giro de 90° en sentido antihorario, equivalente a un giro de 270° en sentido horario [14] [9].

Desde el punto de vista de la codificación, esta caracterización estructural tiene una consecuencia directa: no es adecuado representar el cubo *fuse* simplemente como un cubo clásico con menos piezas visibles. La presencia del bloque fusionado modifica tanto el conjunto de componentes móviles como la semántica de los giros. Por ello, la infraestructura computacional del problema debe construirse a partir de una re-

presentación específica, capaz de distinguir entre piezas móviles e inmutables y de reflejar el efecto de los movimientos legales sobre las primeras.

En particular, la descripción del estado deberá recoger, para cada una de las 7 esquinas y 9 aristas móviles, dos tipos de información: su posición actual dentro del cubo y su orientación relativa. Esta forma de modelado enlaza con la representación basada en *cubies*, utilizada en la resolución automática del cubo por su eficiencia para manipular estados internos [8].

3.2. Sistema de referencia espacial y notación

Una vez definida la estructura física del cubo *fuse*, el siguiente paso consiste en establecer un sistema de referencia espacial y una notación de movimientos. Esto es necesario porque la descripción de posiciones, orientaciones y operadores depende del marco de referencia utilizado. Si dicho marco no se fija desde el principio, la representación computacional puede volverse ambigua, especialmente en un cubo asimétrico como el *fuse*, donde la presencia del bloque rígido rompe la equivalencia geométrica entre unas caras y otras.

En el cubo de Rubik clásico, la simetría del rompecabezas permite cambiar la orientación inicial sin modificar la naturaleza general del problema. Sin embargo, en el cubo *fuse* esta simetría queda condicionada por la pieza rígida $2 \times 2 \times 2$. Por ello, en este trabajo se adopta una orientación fija del cubo, que se mantiene constante durante todo el desarrollo del modelo.

La orientación base elegida es la siguiente: la cara frontal se identifica con el color blanco, la cara superior con azul, la cara derecha con rojo, la cara izquierda con naranja, la cara inferior con verde y la cara trasera con amarillo. Bajo esta convención, el bloque rígido $2 \times 2 \times 2$ queda situado en la región superior-derecha-frontal del cubo. Esta elección permite fijar desde el inicio las direcciones *up*, *right* y *front*, evitando re-interpretaciones posteriores del estado. Además, facilita la traducción de la geometría real del rompecabezas a una codificación computacional basada en piezas móviles.

A partir de esta orientación, las caras del cubo se nombran mediante la convención estándar:

$$U \text{ (Up)}, \quad D \text{ (Down)}, \quad L \text{ (Left)}, \quad R \text{ (Right)}, \quad F \text{ (Front)}, \quad B \text{ (Back)}.$$

Esta nomenclatura coincide con la utilizada habitualmente en el cubo de Rubik y en la normativa de la World Cube Association, donde cada letra identifica un giro elemental de la cara correspondiente [14]. Aunque dicha notación se emplea normalmente para el cubo $3 \times 3 \times 3$ convencional, también resulta válida en este trabajo siempre que se tenga en cuenta que, en el cubo *fuse*, no todas las caras conservan libertad de giro.

Conviene señalar que, en esta memoria, la notación de movimientos describe únicamente operadores sobre el estado del cubo y no incluye rotaciones globales del rompecabezas. Es decir, no se consideran movimientos del tipo *x*, *y* o *z*, utilizados en otras formulaciones para representar reorientaciones completas del cubo. La razón es que la orientación espacial se mantiene fija durante todo el desarrollo.

Una vez fijadas las caras, es posible definir la nomenclatura de las posiciones ocupadas por las piezas. Las posiciones de esquina se describen mediante ternas de caras,

como UFL , ULB o DRB , mientras que las posiciones de arista se representan mediante pares de caras, como UL , DF o BR . Esta convención vincula cada posición con las caras que concurren en ella, lo que facilita tanto la descripción geométrica del cubo como su implementación interna.

En el caso del cubo *fuse*, esta nomenclatura permite distinguir las posiciones móviles de aquellas que forman parte del bloque $2 \times 2 \times 2$. Bajo la orientación adoptada, el bloque fijo se identifica con la región URF , junto con las aristas UR , UF y FR , y los centros U , R y F . Por tanto, estas posiciones no se tratan como piezas móviles independientes dentro del modelo.

3.3. Codificación de las piezas móviles

Una vez fijada la estructura general del cubo *fuse* y el sistema de referencia espacial empleado en este trabajo, el siguiente paso consiste en determinar qué elementos del rompecabezas deben representarse explícitamente dentro del modelo computacional.

En el cubo de Rubik $3 \times 3 \times 3$ convencional, el conjunto de piezas móviles está formado por 8 esquinas y 12 aristas, mientras que los 6 centros permanecen fijos entre sí y actúan como referencia de color. En el cubo *fuse*, la situación es distinta debido a la presencia de una pieza rígida $2 \times 2 \times 2$, que integra en un único bloque varias subpiezas que en el cubo clásico serían independientes. Como consecuencia, estas piezas no deben codificarse como unidades móviles autónomas.

Bajo la convención adoptada, el bloque rígido ocupa la región superior-derecha-frontal del cubo. Desde un punto de vista geométrico, esto implica que la esquina correspondiente a dicha región, junto con tres aristas adyacentes, queda absorbida por la pieza rígida y deja de formar parte del conjunto de *cubies* móviles. En consecuencia, el cubo *fuse* queda compuesto por:

- 7 esquinas móviles,
- 9 aristas móviles,
- y un bloque rígido $2 \times 2 \times 2$ que permanece invariante.

3.3.1. Esquinas móviles

Las esquinas son las piezas del rompecabezas en las que confluyen tres caras. En el cubo clásico existen 8 posiciones de esquina, pero en el cubo *fuse* una de ellas queda integrada en el bloque rígido y deja de tratarse como una unidad independiente. Por tanto, el modelo solo necesita representar 7 esquinas móviles. Utilizando la nomenclatura basada en ternas de caras introducida en la subsección anterior, las posiciones de esquina móviles se definen como:

$$\mathcal{C} = \{UFL, ULB, UBR, DFR, DLF, DBL, DRB\}.$$

La posición URF no aparece en este conjunto porque forma parte de la pieza fusionada. Al estar físicamente unida al bloque rígido, dicha esquina no puede permutarse de forma independiente ni necesita almacenarse como un *cubie* separado dentro del estado.

3.3. Codificación de las piezas móviles

Para cada una de las 7 esquinas móviles no basta con conocer qué pieza ocupa una determinada posición. También es necesario conocer su orientación. Una esquina puede encontrarse en la posición correcta, pero con una orientación incorrecta respecto a los colores de las caras adyacentes. Por ello, cada esquina se codifica mediante dos componentes:

1. su **permutación**, es decir, qué pieza de esquina ocupa una posición dada;
2. su **orientación**, que indica cómo está rotada dicha pieza dentro de esa posición.

Desde el punto de vista formal, la orientación de una esquina puede representarse mediante un valor entero módulo 3. Esta convención es habitual en las representaciones basadas en *cubies*, ya que una esquina tiene tres orientaciones posibles compatibles con su geometría [8, 15]. En términos prácticos, esto significa que a cada esquina se le puede asociar un valor perteneciente al conjunto:

$$\{0, 1, 2\},$$

donde el valor 0 representa la orientación de referencia y los valores 1 y 2 representan las dos rotaciones posibles restantes. La interpretación exacta de cada valor depende de la convención elegida para leer la orientación, pero a nivel estructural lo importante es que la orientación de esquina se trata como una variable ternaria.

3.3.2. Aristas móviles

Las aristas son las piezas en las que confluyen dos caras. En el cubo $3 \times 3 \times 3$ ordinario existen 12 posiciones de arista, pero en el cubo *fuse* tres de ellas quedan absorbidas por el bloque rígido. En consecuencia, el modelo solo representa 9 aristas móviles. Empleando la notación basada en pares de caras, dichas posiciones se definen como:

$$\mathcal{E} = \{UL, UB, DR, DF, DL, DB, FL, BL, BR\}.$$

Las posiciones UR , UF y FR no aparecen en esta colección porque quedan integradas en el bloque $2 \times 2 \times 2$. Al igual que ocurre con la esquina fija, estas aristas no se consideran entidades independientes en la infraestructura computacional del cubo, ya que su comportamiento queda determinado por el bloque rígido al que pertenecen.

De igual manera que en el caso de las esquinas, cada arista móvil se representa mediante dos tipos de información:

1. su **permutación**, que indica qué pieza de arista ocupa una posición concreta;
2. su **orientación**, que refleja si la pieza se encuentra correctamente alineada o invertida respecto a las caras de referencia.

A diferencia de las esquinas, la orientación de una arista solo presenta dos posibilidades geométricas, por lo que puede codificarse mediante un valor entero módulo 2:

$$\{0, 1\}.$$

El valor 0 se interpreta como orientación correcta según la convención de referencia, mientras que el valor 1 representa una inversión de la arista. Esta codificación binaria es habitual en los modelos internos del cubo de Rubik, donde la orientación de las aristas se representa mediante dos valores posibles [8, 15].

3.3.3. Bloque rígido e información invariante

Junto a las piezas móviles, el modelo incorpora una pieza especial: el bloque rígido $2 \times 2 \times 2$. A diferencia del resto de elementos, este bloque no se codifica mediante variables de permutación ni de orientación independientes, ya que su papel dentro del modelo es el de una restricción estructural fija. Es decir, el bloque no forma parte de la dinámica del estado, sino que actúa como una condición geométrica permanente que limita qué movimientos son posibles y qué piezas pueden cambiar de posición.

Desde el punto de vista del software, no es necesario almacenar explícitamente el estado interno del bloque en cada configuración. Basta con conocer su localización de referencia y utilizar esa información para:

- excluir de la representación las piezas que pertenecen al bloque,
- restringir el conjunto de movimientos legales,
- y mantener una interpretación coherente de la geometría del cubo durante la ejecución.

Esta forma de proceder simplifica la representación. En lugar de modelar el bloque rígido como una agregación de piezas parcialmente móviles, se asume como una entidad invariante del sistema. Así, la infraestructura computacional queda centrada únicamente en las piezas que participan en la evolución del rompecabezas.

3.3.4. Identidad de pieza frente a posición

Un aspecto que conviene aclarar antes de definir la representación completa del estado es la distinción entre *pieza* y *posición*. En un cubo combinatorio, ambas nociones no deben confundirse. La **pieza** hace referencia al *cubie* físico concreto, identificado por su combinación de colores o por su identidad dentro del conjunto de piezas móviles. La **posición**, en cambio, designa el lugar geométrico que una pieza ocupa dentro del cubo en un instante dado.

Esta diferencia es importante porque el estado del rompecabezas no se describe únicamente indicando qué posiciones existen, sino especificando qué pieza ocupa cada posición y con qué orientación lo hace. Por ejemplo, una posición de esquina como *ULB* puede estar ocupada por la pieza *DBL*, y además dicha pieza puede encontrarse girada respecto a su orientación de referencia. Por tanto, la infraestructura computacional debe ser capaz de almacenar simultáneamente:

- la correspondencia entre posiciones y piezas;
- y la orientación asociada a cada pieza en su posición actual.

Esta separación es la base de la representación mediante permutaciones y orientaciones, utilizada en modelos basados en *cubies*. En este trabajo, dicha idea se adapta al cubo *fuse*, reduciendo el marco de piezas al conjunto $\mathcal{C} \cup \mathcal{E}$ de esquinas y aristas móviles, mientras que el bloque rígido se trata como una restricción fija del modelo.

3.3.5. Justificación de la codificación elegida

La decisión de codificar el cubo *fuse* a partir de sus piezas móviles, y no a partir de una representación basada únicamente en pegatinas o colores visibles, responde a criterios teóricos y prácticos. Desde el punto de vista teórico, la representación por *cubies* permite describir la dinámica del rompecabezas mediante piezas y orientaciones, no mediante colores aislados. Desde el punto de vista práctico, esta elección simplifica la implementación de operadores y la validación de estados, puesto que cada giro puede definirse como una transformación discreta sobre un conjunto finito de posiciones.

Además, este enfoque resulta adecuado para su integración posterior en un EDA, ya que permite construir funciones de transición eficientes y evita inconsistencias derivadas de representaciones más superficiales. En el cubo *fuse*, donde la presencia del bloque rígido introduce una restricción adicional, disponer de una representación interna clara es especialmente importante para el desarrollo del solver.

3.4. Representación del estado del cubo

Una vez identificadas las piezas móviles del cubo *fuse* y fijado el sistema de referencia espacial, el siguiente paso consiste en definir cómo se representa un estado del rompecabezas dentro del modelo computacional. Esta cuestión es central en la infraestructura del cubo, ya que cualquier algoritmo que opere sobre él necesita una descripción interna precisa, compacta y manipulable de sus configuraciones. La representación elegida debe permitir tres operaciones fundamentales: almacenar un estado, transformar dicho estado mediante un movimiento legal y comprobar si la configuración resultante coincide con el estado objetivo.

En problemas relacionados con el cubo de Rubik, una de las decisiones más importantes es elegir entre una representación basada en *facelets* o pegatinas visibles y una representación basada en *cubies*, es decir, en piezas móviles. La primera describe el color presente en cada posición superficial del rompecabezas; la segunda describe directamente qué pieza ocupa cada posición y con qué orientación lo hace. En la literatura sobre solvers del cubo, la segunda opción es habitual cuando el objetivo es manipular estados de forma eficiente, ya que los operadores actúan de forma natural sobre piezas y no sobre pegatinas individuales. Tanto la representación a nivel de *cubies* explicada por Kociemba como otros trabajos de resolución automática del cubo parten de esta idea de separar la permutación y la orientación de esquinas y aristas [8, 15].

En este trabajo se adopta este enfoque, adaptándolo a la estructura particular del cubo *fuse*. Por tanto, un estado se modela como la combinación de cuatro componentes:

$$S = (cp, co, ep, eo),$$

donde:

- *cp* representa la **permutación de esquinas**,
- *co* representa la **orientación de esquinas**,
- *ep* representa la **permutación de aristas**,

- eo representa la **orientación de aristas**.

Esta descomposición refleja que, para describir completamente una configuración, no basta con saber qué piezas existen, sino que es necesario indicar en qué posición se encuentra cada una y cómo está orientada dentro de esa posición. De esta manera, el estado global del cubo queda reducido a un conjunto finito de datos discretos, fácilmente tratables desde el punto de vista algorítmico.

3.4.1. Conjuntos de posiciones de referencia

Para formalizar esta representación, se consideran dos conjuntos ordenados de posiciones: uno para las esquinas móviles y otro para las aristas móviles. Siguiendo la orientación y la nomenclatura fijadas anteriormente, las posiciones de esquina se representan mediante:

$$\mathcal{C} = \{UFL, ULB, UBR, DFR, DLF, DBL, DRB\},$$

mientras que las posiciones de arista se representan mediante:

$$\mathcal{E} = \{UL, UB, DR, DF, DL, DB, FL, BL, BR\}.$$

Estos dos conjuntos no solo identifican qué posiciones son relevantes para el modelo, sino que además establecen un orden fijo sobre ellas. Este detalle es importante porque la representación computacional del estado se implementa mediante vectores indexados. En consecuencia, la posición que ocupa cada componente dentro del vector tiene un significado geométrico estable durante toda la ejecución. Por ejemplo, el primer elemento del vector de esquinas corresponde siempre a la posición UFL , el segundo a ULB , y así sucesivamente. Lo mismo ocurre con las aristas.

La imposición de un orden fijo sobre $\mathcal{C}$ y $\mathcal{E}$ permite transformar la estructura espacial del cubo en un vector simple formado por valores enteros.

3.4.2. Permutación de esquinas

La componente cp (*corner permutation*) describe qué pieza de esquina ocupa cada una de las posiciones del conjunto $\mathcal{C}$. Formalmente, puede entenderse como una permutación del conjunto de las 7 esquinas móviles. Si se numera cada esquina mediante un identificador entero del 0 al 6, entonces cp puede representarse como un vector:

$$cp = (cp_0, cp_1, \dots, cp_6),$$

donde cp_i indica qué pieza de esquina se encuentra en la i -ésima posición de $\mathcal{C}$.

Por ejemplo, si la esquina que en el estado resuelto pertenece a la posición DBL se encontrase ocupando la posición UFL , entonces el primer componente de cp contendría el identificador asociado a DBL . De este modo, el vector no codifica colores ni pegatinas visibles, sino la ocupación efectiva de cada posición por una pieza concreta.

Este tipo de representación coincide con el modelo a nivel de *cubies*, en el que la permutación de esquinas se separa explícitamente de su orientación. La ventaja principal es que cada movimiento del rompecabezas puede describirse como una transformación discreta de esta permutación, lo que simplifica la aplicación de operadores y la comparación entre estados [8, 15].

3.4.3. Orientación de esquinas

La componente *co* (*corner orientation*) almacena la orientación de cada una de las piezas de esquina en su posición actual. Como ya se indicó en la subsección anterior, una esquina posee tres orientaciones geoméricamente posibles, por lo que cada una puede representarse mediante un valor entero módulo 3:

$$co_i \in \{0, 1, 2\}.$$

En consecuencia, la orientación global de las esquinas puede escribirse como:

$$co = (co_0, co_1, \dots, co_6),$$

donde cada co_i indica la orientación de la esquina situada en la i -ésima posición de $\mathcal{C}$.

La interpretación de estos valores depende de una convención de referencia. En el modelo a nivel de *cubies*, dicha convención suele definirse a partir de referencias asociadas a las posiciones del cubo, de manera que la orientación 0 representa la alineación correcta respecto al estado resuelto, mientras que los valores 1 y 2 representan las dos torsiones posibles de la esquina. Esta idea aparece en la formalización de Kociemba, donde la orientación de esquinas y aristas se trata como una componente independiente de la permutación [8, 15].

En el contexto concreto del cubo *fuse*, la información de orientación sigue siendo necesaria porque una esquina puede encontrarse en la posición adecuada y, sin embargo, estar girada incorrectamente. Por tanto, la permutación por sí sola no basta para decidir si una configuración está resuelta ni para actualizar correctamente el estado tras aplicar un giro que modifique la orientación de ciertas esquinas.

3.4.4. Permutación de aristas

De forma análoga, la componente *ep* (*edge permutation*) indica qué pieza de arista ocupa cada posición del conjunto $\mathcal{E}$. Si se numeran las 9 aristas móviles mediante identificadores del 0 al 8, entonces:

$$ep = (ep_0, ep_1, \dots, ep_8),$$

donde ep_i es la pieza de arista situada en la i -ésima posición de $\mathcal{E}$.

Esta componente describe exclusivamente la localización de las aristas, sin incorporar información sobre si están correctamente alineadas o invertidas. Igual que sucede con las esquinas, la ventaja de esta separación es que permite tratar la acción de un movimiento como una transformación estructurada sobre datos discretos. Un giro puede permutar varias aristas a la vez, pero no necesariamente altera su orientación de la misma manera; por ello, es conveniente mantener ambas nociones en variables distintas.

La representación mediante permutaciones de aristas es estándar dentro del modelo a nivel de *cubies*, tanto en desarrollos teóricos como en implementaciones de solvers. En dicho contexto, la permutación de aristas constituye una de las partes esenciales del estado interno del cubo [8, 15].

3.4.5. Orientación de aristas

La cuarta componente del estado es eo (*edge orientation*), que describe la orientación de las aristas móviles. A diferencia de las esquinas, cada arista solo admite dos posibilidades geométricas: orientación correcta o inversión. Por ello, cada componente de orientación de arista puede representarse mediante un valor binario:

$$eo_i \in \{0, 1\}.$$

Así, el vector global de orientación de aristas toma la forma:

$$eo = (eo_0, eo_1, \dots, eo_8).$$

De nuevo, el valor 0 representa la orientación de referencia y el valor 1 una inversión de la arista respecto a esa referencia. En la formulación clásica del cubo a nivel de *cubies*, esta separación entre permutación y orientación permite construir coordenadas y tablas de movimiento eficientes. En particular, Kociemba muestra cómo las orientaciones de esquinas y aristas se prestan a una codificación numérica compacta, al tratarse de variables ternarias y binarias, respectivamente [8, 15].

En el cubo *fuse*, aunque el número total de aristas consideradas es menor que en el cubo estándar, la lógica de representación se mantiene. Cada arista móvil necesita una componente de posición y una componente de orientación para capturar correctamente la evolución del rompecabezas ante los movimientos legales.

3.4.6. Estado resuelto

Una vez definidas las cuatro componentes del estado, puede caracterizarse formalmente la configuración resuelta del cubo. Se considera que el cubo *fuse* está resuelto cuando cada pieza móvil se encuentra en su posición de referencia y con orientación correcta. Si se adopta la numeración inducida por el orden de $\mathcal{C}$ y $\mathcal{E}$, entonces el estado resuelto queda dado por:

$$cp = (0, 1, 2, 3, 4, 5, 6),$$

$$co = (0, 0, 0, 0, 0, 0, 0),$$

$$ep = (0, 1, 2, 3, 4, 5, 6, 7, 8),$$

$$eo = (0, 0, 0, 0, 0, 0, 0, 0, 0).$$

Esta definición proporciona un criterio computacional directo para reconocer la solución: basta con comprobar la igualdad exacta entre el estado actual y el estado resuelto de referencia. De este modo, el objetivo del rompecabezas puede expresarse de forma algebraica, sin necesidad de inspeccionar visualmente la disposición de colores sobre las caras.

Representación óptima para el EDA

Se ha elegido esta representación basada en *cubies* porque un EDA es un método poblacional: en cada generación se muestrean cientos de individuos, y cada individuo codifica una cadena de movimientos que debe ejecutarse paso a paso desde el

estado mezclado para poder evaluarla. Esto implica que, a lo largo de la ejecución completa del solver, el número de transiciones de estado y de evaluaciones intermedias asciende a cifras muy elevadas. La representación por cubies hace que estas operaciones sean muy baratas, ya que se reducen a reindexaciones de posiciones y sumas modulares sobre cuatro vectores cortos de enteros, sin estructuras complejas ni interpretación de colores, lo que dota al solver de mayor eficiencia y rapidez de ejecución.

3.5. Modelado de la acción de cada movimiento

Una vez fijado el espacio de movimientos permitido del cubo *fuse*, el siguiente paso consiste en describir cómo actúa cada uno de esos movimientos sobre la representación interna del estado. El modelo del rompecabezas no queda definido únicamente indicando cuáles son los operadores legales; también es necesario especificar qué transformación produce cada operador sobre las permutaciones y orientaciones de las piezas móviles.

Recordemos que, en este trabajo, un estado del cubo *fuse* se representa mediante la cuádrupla:

$$S = (cp, co, ep, eo),$$

donde *cp* y *co* representan respectivamente la permutación y la orientación de las esquinas móviles, mientras que *ep* y *eo* representan la permutación y la orientación de las aristas móviles. Por tanto, modelar la acción de un movimiento equivale a definir una función de transición que, dada una configuración *S*, devuelva una nueva configuración *S'* en la que se hayan actualizado las cuatro componentes anteriores de acuerdo con el giro aplicado.

3.5.1. Función de transición de estados

Sea $m \in \mathcal{M}$ un movimiento legal del cubo *fuse*. La acción de *m* sobre un estado *S* puede formalizarse mediante una función de transición:

$$T_m : \mathcal{S} \longrightarrow \mathcal{S}, \quad T_m(S) = S'.$$

En esta expresión, *S* denota el conjunto de estados válidos del cubo y *S'* representa el estado obtenido tras aplicar el movimiento *m* sobre *S*.

Desde el punto de vista computacional, cada movimiento se implementa como una combinación de dos tipos de operaciones:

1. una **permutación de posiciones**, que redistribuye las piezas afectadas por el giro;
2. una posible **actualización de orientación**, que modifica la orientación de aquellas piezas cuya geometría así lo requiera.

Esta separación entre permutación y orientación es coherente con la representación a nivel de *cubies*, donde cada operador actúa como una transformación discreta sobre las esquinas y aristas del rompecabezas [8, 15]. La ventaja de este enfoque es que permite describir la dinámica del cubo mediante reglas algebraicas, sin necesidad de trabajar directamente con colores o pegatinas visibles.

3.5.2. Convención de indexación

Para expresar formalmente la acción de cada movimiento, se parte de los conjuntos ordenados de posiciones introducidos en la subsección anterior:

$$\mathcal{C} = \{UFL, ULB, UBR, DFR, DLF, DBL, DRB\},$$

$$\mathcal{E} = \{UL, UB, DR, DF, DL, DB, FL, BL, BR\}.$$

El orden fijado sobre $\mathcal{C}$ y $\mathcal{E}$ permite numerar las posiciones de las esquinas mediante índices $0, \dots, 6$ y las posiciones de las aristas mediante índices $0, \dots, 8$. Gracias a ello, la acción de un movimiento puede expresarse mediante vectores de permutación y vectores de actualización de orientación.

En concreto, para cada movimiento base $m \in \{L, D, B\}$ se definen:

- una permutación de esquinas π_m^C ,
- una variación de orientación de esquinas δ_m^C ,
- una permutación de aristas π_m^E ,
- una variación de orientación de aristas δ_m^E .

La convención adoptada es la siguiente: el valor $\pi_m^C(i)$ indica qué posición de esquina previa pasa a ocupar la posición i tras aplicar el movimiento. Del mismo modo, $\pi_m^E(j)$ indica qué posición de arista previa pasa a ocupar la posición j en el nuevo estado. Esta elección se ajusta a una implementación mediante vectores indexados, ya que cada posición destino toma la pieza procedente de una posición origen determinada.

Con esta convención, si

$$S = (cp, co, ep, eo)$$

es el estado actual y

$$S' = (cp', co', ep', eo')$$

es el estado resultante tras aplicar m , entonces las componentes del nuevo estado vienen dadas por:

$$cp'_i = cp_{\pi_m^C(i)},$$

$$co'_i = (co_{\pi_m^C(i)} + \delta_m^C(i)) \text{ mód } 3,$$

$$ep'_j = ep_{\pi_m^E(j)},$$

$$eo'_j = (eo_{\pi_m^E(j)} + \delta_m^E(j)) \text{ mód } 2.$$

Estas expresiones recogen la información necesaria para modelar un giro: la pieza que llega a cada posición y la posible modificación de su orientación una vez situada en ella.

3.5.3. Acción del movimiento L

El movimiento L corresponde a un giro de 90° de la cara izquierda. Bajo la orientación adoptada en este trabajo, dicho giro afecta a las esquinas y aristas adyacentes a la cara L que no pertenecen al bloque rígido.

En términos geométricos, L actúa sobre las esquinas:

$$UFL, ULB, DBL, DLF,$$

y sobre las aristas:

$$UL, BL, DL, FL.$$

En la convención de indexación empleada, la acción de L sobre las esquinas se representa mediante:

$$\pi_L^C = (1, 5, 2, 3, 0, 4, 6),$$

$$\delta_L^C = (1, 2, 0, 0, 2, 1, 0).$$

Análogamente, sobre las aristas:

$$\pi_L^E = (7, 1, 2, 3, 6, 5, 0, 4, 8),$$

$$\delta_L^E = (0, 0, 0, 0, 0, 0, 0, 0, 0).$$

Por ejemplo, el primer componente de π_L^C indica que, tras aplicar L , la posición UFL pasa a estar ocupada por la esquina que antes se encontraba en ULB . Del mismo modo, la posición DLF pasa a recibir la esquina que anteriormente estaba en UFL .

Desde un punto de vista geométrico, esto equivale a una permutación cíclica de las esquinas de la cara izquierda. Si se describe el movimiento en términos de la trayectoria seguida por las piezas, las esquinas afectadas recorren el ciclo:

$$UFL \rightarrow DLF \rightarrow DBL \rightarrow ULB \rightarrow UFL.$$

En el caso de las aristas, el ciclo inducido por el movimiento L es:

$$UL \rightarrow FL \rightarrow DL \rightarrow BL \rightarrow UL.$$

Además de permutar dichas piezas, el giro L modifica la orientación de las esquinas implicadas según el vector δ_L^C . Sin embargo, en la convención adoptada no altera la orientación de las aristas afectadas, por lo que δ_L^E es el vector nulo.

3.5.4. Acción del movimiento D

El movimiento D representa un giro de 90° de la cara inferior. Dado que esta cara no se encuentra absorbida por el bloque rígido, el operador es legal dentro del cubo *fuse* considerado en este trabajo.

Geométricamente, D afecta a las esquinas:

$$DFR, DLF, DBL, DRB,$$

y a las aristas:

$$DR, DF, DL, DB.$$

La acción de D sobre las esquinas se representa mediante:

$$\pi_D^C = (0, 1, 2, 4, 5, 6, 3),$$

$$\delta_D^C = (0, 0, 0, 0, 0, 0, 0),$$

mientras que sobre las aristas viene dada por:

$$\pi_D^E = (0, 1, 3, 4, 5, 2, 6, 7, 8),$$

$$\delta_D^E = (0, 0, 0, 0, 0, 0, 0, 0, 0).$$

En consecuencia, el movimiento D permuta las piezas de la capa inferior sin alterar la orientación ni de las esquinas ni de las aristas. En términos de trayectoria de piezas, puede describirse mediante los ciclos:

$$DFR \rightarrow DRB \rightarrow DBL \rightarrow DLF \rightarrow DFR,$$

$$DR \rightarrow DB \rightarrow DL \rightarrow DF \rightarrow DR.$$

Este comportamiento convierte a D en el operador más simple de los tres movimientos base del modelo, ya que su acción consiste únicamente en redistribuir piezas entre posiciones de la capa inferior, sin producir torsiones ni inversiones adicionales. Desde el punto de vista de la implementación, basta con actualizar las permutaciones cp y ep , manteniendo invariantes co y eo .

3.5.5. Acción del movimiento B

El movimiento B corresponde a un giro de 90° de la cara trasera. En la orientación elegida para el cubo *fuse*, esta cara tampoco queda bloqueada por la pieza rígida, por lo que forma parte del conjunto de operadores permitidos.

El giro B afecta a las esquinas:

$$ULB, UBR, DRB, DBL,$$

y a las aristas:

$$UB, BR, DB, BL.$$

Bajo la convención empleada, la acción de B sobre las esquinas se modela mediante:

$$\pi_B^C = (0, 2, 6, 3, 4, 1, 5),$$

$$\delta_B^C = (0, 1, 2, 0, 0, 2, 1),$$

mientras que su acción sobre las aristas viene dada por:

$$\pi_B^E = (0, 8, 2, 3, 4, 7, 6, 1, 5),$$

$$\delta_B^E = (0, 1, 0, 0, 0, 1, 0, 1, 1).$$

Interpretando estas expresiones en términos geométricos, el movimiento B induce en las esquinas el ciclo:

$$ULB \rightarrow DBL \rightarrow DRB \rightarrow UBR \rightarrow ULB,$$

y en las aristas el ciclo:

$$UB \rightarrow BL \rightarrow DB \rightarrow BR \rightarrow UB.$$

A diferencia de D , el movimiento B sí altera la orientación de parte de las piezas afectadas. En concreto, las esquinas implicadas modifican su orientación de acuerdo con δ_B^C , y las aristas de la cara trasera invierten su orientación según el patrón indicado por δ_B^E . Esta actualización permite que la representación capture el efecto del giro trasero sobre el estado interno del cubo.

3.5.6. Movimientos derivados

Aunque la infraestructura se apoya en los tres movimientos base L , D y B , el conjunto completo de operadores del cubo *fuse* incluye también las medias vueltas y los giros inversos. Estos movimientos no requieren una definición independiente, ya que pueden obtenerse por composición de los movimientos base:

$$\begin{aligned} L^2 &= L \circ L, & L' &= L \circ L \circ L, \\ D^2 &= D \circ D, & D' &= D \circ D \circ D, \\ B^2 &= B \circ B, & B' &= B \circ B \circ B. \end{aligned}$$

Desde el punto de vista matemático, esta formulación implica que el comportamiento de todo el alfabeto de movimientos queda determinado por las funciones de transición asociadas a L , D y B . Desde el punto de vista de la implementación, permite reutilizar las mismas transformaciones elementales y construir el resto de operadores mediante aplicación repetida.

Esta estrategia ayuda a mantener la coherencia interna del modelo. Al derivar L^2 , L' , D^2 , D' , B^2 y B' a partir de las mismas definiciones base, se evita introducir diferencias entre la interpretación geométrica de un giro y su implementación computacional.

3.5.7. Consistencia del modelo de transición

La definición anterior establece la dinámica del cubo *fuse*. Cada movimiento legal queda descrito mediante una transformación determinista sobre las cuatro componentes del estado, y cada una de estas transformaciones conserva la validez de la configuración resultante. En otras palabras, si el estado inicial pertenece al espacio de estados válidos S , entonces la aplicación de cualquier movimiento legal produce otro estado de S .

Esta propiedad asegura que la infraestructura computacional no solo describe configuraciones aisladas del rompecabezas, sino también la forma en la que dichas configuraciones se conectan entre sí. Así, el cubo *fuse* queda modelado como un sistema discreto cuya evolución viene gobernada por un conjunto finito de funciones de transición.

En definitiva, la acción de los movimientos sobre el cubo *fuse* puede formalizarse mediante permutaciones de posiciones y actualizaciones modulares de orientación. Los operadores base L , D y B determinan la dinámica del modelo, y el resto de movimientos permitidos se obtienen como composiciones de estos. Con ello queda definida la infraestructura dinámica del cubo, al disponer de una descripción formal del estado y de la forma en que cada movimiento lo transforma.

Capítulo 4

Diseño en Implementación del Solver basado en EDAs

Una vez modelada la infraestructura del cubo Fuse en el capítulo 3, queda por concretar qué variante de algoritmo de estimación de distribuciones se aplicará sobre dicho modelo y con qué decisiones de diseño se construirá el solver. Este capítulo aborda la elección concreta del EDA, la codificación del problema como genoma, la función de evaluación que guía la selección, la inicialización del modelo probabilístico y la estrategia de exploración mediante reinicios.

4.1. Justificación del modelo EDA elegido

La familia de los EDA se subdivide habitualmente en función del tipo de variables que codifica el genoma y de la complejidad del modelo probabilístico empleado, tal como se describió en la subsección 2.2.3. Ambos aspectos condicionan la elección que se realiza en este trabajo.

Respecto al primer aspecto, en el solver desarrollado un individuo se representa como una secuencia ordenada de movimientos legales del cubo, donde cada posición del genoma toma valor en un alfabeto finito y discreto formado por los nueve movimientos permitidos del cubo Fuse junto con un token especial STOP. Como se utilizan genomas categóricos, se descartan de partida las variantes de EDA diseñadas para variables continuas, como la versión continua de UMDA ($UMDA_c$, basada en distribuciones gaussianas) o EGNA, pensadas para optimización en dominios reales [16]. La variante que se utiliza en este trabajo es $UMDA_{cat}$, la versión categórica del mismo algoritmo, en la que cada posición del genoma se modela mediante una distribución sobre un conjunto discreto de valores posibles.

Respecto al segundo aspecto, dentro de los EDA categóricos la decisión está entre modelos sin dependencias o modelos más complejos basados en redes bayesianas como EBNA o BOA. En este trabajo se opta por un modelo univariado categórico, implementado mediante la clase $UMDA_{cat}$ de EDAspy. Este modelo se basa en la idea general del algoritmo UMDA, introducido por Mühlenbein y Paaß [17], pero adaptado al caso de variables categóricas discretas mediante la implementación proporcionada por EDAspy [16]. La justificación de esta elección es triple. En primer lugar, el modelo univariado es el más simple y rápido de ajustar, ya que trata cada posición del geno-

4.1. Justificación del modelo EDA elegido

ma por separado, sin estimar relaciones entre posiciones. En segundo lugar, el coste de ajustar un modelo más complejo como una red bayesiana crece de forma notable con la longitud del genoma y con el tamaño de la población, lo que para genomas de entre 18 y 30 posiciones y poblaciones de varios cientos de individuos resultaría demasiado costoso computacionalmente. En tercer lugar, trabajos teóricos recientes han demostrado que UMDA, con una elección adecuada del tamaño de población, es capaz de manejar problemas difíciles incluso cuando el fitness resulta engañoso [18], por lo que su simplicidad no impide abordar problemas complejos como el del cubo.

La principal limitación del modelo univariado se asume de forma explícita: las posiciones del genoma del cubo Fuse no son independientes en la realidad. Movimientos sobre la misma cara en posiciones contiguas son redundantes ($L L'$ se cancela y $L L$ se simplifica a L^2), y existen correlaciones claras entre posiciones consecutivas que un modelo univariado no puede capturar. Para compensar esta carencia, se añade conocimiento del problema en otros componentes del solver: la simplificación de cancelaciones aplicada tras el muestreo (subsección 3.2.2), la penalización de cara repetida y el sesgo por el inverso del scramble en la inicialización (subsección 3.2.4), y las demás decisiones de diseño que se detallan más adelante.

La implementación se apoya en la biblioteca EDAspy [16], una librería de Python desarrollada por el Computational Intelligence Group de la Universidad Politécnica de Madrid. EDAspy proporciona la clase `UMDAcat`, que implementa el algoritmo UMDA para variables categóricas y expone los parámetros principales que regulan su comportamiento, como el tamaño de la población, el número máximo de iteraciones, la fracción de selección o la tabla de frecuencias inicial. El uso de una biblioteca consolidada permite centrar el esfuerzo del trabajo en las decisiones específicas del problema como pueden ser: la función de evaluación, sesgos de inicialización y estrategia de reinicios, en lugar de en la implementación del algoritmo base.

La Figura 4.1 resume el funcionamiento general del solver propuesto. A partir de un scramble inicial, el algoritmo genera una población de secuencias candidatas, las evalúa mediante la función de fitness definida para el cubo *fuse*, selecciona los mejores individuos y actualiza un modelo probabilístico univariado para producir nuevas soluciones.

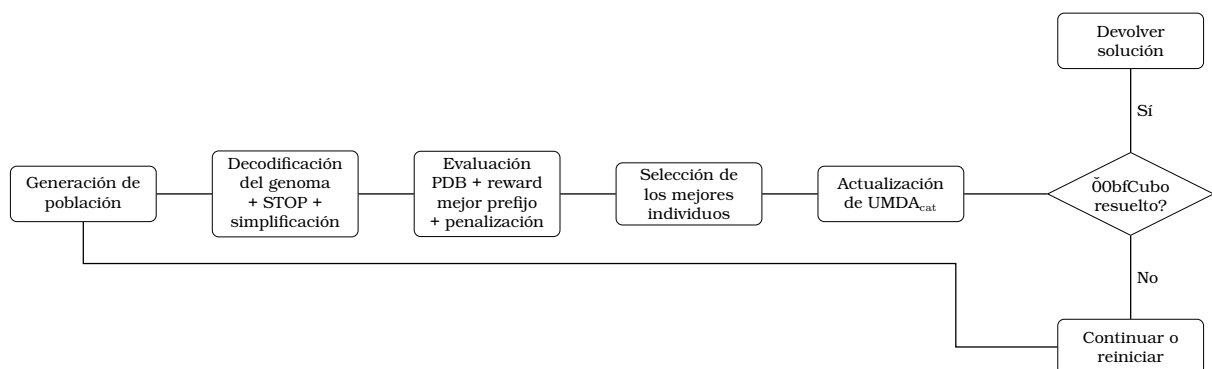


Figura 4.1: Flujo general del solver basado en UMDA_{cat}.

4.2. Representación del problema como genoma

Una vez justificado el modelo EDA, es necesario definir cómo se codifica un candidato a solución. En el solver desarrollado, un individuo del EDA es una secuencia ordenada de movimientos del cubo. Esta elección es natural: la salida que se espera del solver es precisamente una secuencia de giros que, aplicada al cubo mezclado, lo lleve al estado resuelto. La cuestión de diseño no es tanto qué representar, sino con qué alfabeto, con qué longitud y con qué transformaciones intermedias se conecta esa secuencia con la evaluación del individuo.

4.2.1. Alfabeto de genes

Cada posición del genoma toma valor en un alfabeto de diez elementos. Nueve de ellos corresponden a los movimientos legales del cubo Fuse: L , $L2$, L' , D , $D2$, D' , B , $B2$ y B' . El décimo elemento es un token especial, STOP, cuyo papel se explica en la siguiente subsubsección. Un genoma de longitud L es, por tanto, un vector de L posiciones donde cada una puede contener cualquiera de estos diez valores. El valor de L es un hiperparámetro del solver que controla la longitud máxima de la secuencia de movimientos que el algoritmo puede generar, y cuya elección se discutirá en la subsección 3.2.5.

4.2.2. Token STOP y aprendizaje de la longitud efectiva

Un EDA estándar trabaja con genomas de longitud fija. Si se tomase literalmente esa convención, todo individuo del solver sería una secuencia de exactamente L movimientos, independientemente de que la solución real del cubo requiera muchos menos giros. Esto presenta dos problemas: por un lado, fuerza al algoritmo a llenar de movimientos las posiciones sobrantes, lo que genera ruido que destruye estados parcialmente resueltos; por otro, sesga la búsqueda hacia soluciones de longitud fija en lugar de permitir que el algoritmo descubra la longitud que mejor se adapta al scramble concreto.

La introducción del token STOP en el alfabeto resuelve ambos problemas. Durante la decodificación de un individuo, la secuencia de movimientos efectiva se obtiene leyendo el genoma de izquierda a derecha y deteniéndose en la primera aparición de STOP. De este modo, un mismo genoma de longitud nominal L puede dar lugar a secuencias de cualquier longitud entre 0 y L movimientos, según dónde aparezca el primer STOP. La probabilidad que el modelo asigna a STOP en cada posición se convierte así en un parámetro que el algoritmo aprende durante la evolución: si las soluciones buenas son cortas, las posiciones tempranas verán crecer su probabilidad de STOP; si son largas, ocurrirá lo contrario. De esta forma, el EDA dispone de un mecanismo sencillo para descubrir la longitud adecuada de la solución sin necesidad de fijarla de antemano.

4.2.3. Decodificación y simplificación de cancelaciones

La traducción de un genoma en una secuencia de movimientos del cubo se realiza en dos pasos. En el primer paso se recorre el genoma posición por posición, descartando todo lo posterior a la primera aparición de STOP. El resultado es una lista de movimientos que puede contener redundancias.

En el segundo paso se aplica una simplificación que elimina los movimientos consecutivos sobre la misma cara, ya que pueden combinarse en uno solo o cancelarse. Por ejemplo, L seguido de L se simplifica a L^2 ; L seguido de L' se cancela por completo y desaparece de la secuencia; y L^2 seguido de L' se reduce a un único L . La simplificación se aplica de forma iterativa, de modo que las cancelaciones producidas pueden encadenarse: la secuencia (L, L, L', D) se simplifica primero a (L^2, L', D) y luego a (L, D) .

La decisión de aplicar esta simplificación después del muestreo, y no restringirla dentro del modelo, responde a una limitación del modelo univariado. Para evitar que el modelo genere dos movimientos consecutivos en la misma cara haría falta que la probabilidad de una posición dependiese de la anterior, es decir, un modelo que capture dependencias entre posiciones. Como $UMDA_{cat}$ trata cada posición de forma independiente, esa restricción no puede expresarse dentro del modelo y se traslada a esta etapa de post-procesado.

4.2.4. Reconstrucción del algoritmo mediante el mejor prefijo

Una particularidad del solver es que el genoma puede contener movimientos útiles seguidos de movimientos que destruyen el estado alcanzado. Si el algoritmo encuentra una buena solución en los primeros genes pero el modelo aún no ha aprendido a colocar STOP inmediatamente después, las posiciones siguientes contendrán movimientos que alteran el cubo respecto al estado deseado.

Por este motivo, la solución que el solver reporta no es necesariamente la secuencia completa de movimientos, sino el prefijo que produce el mejor estado intermedio. El solver ejecuta la secuencia movimiento a movimiento, evalúa el estado del cubo en cada paso y se queda con el tramo inicial que alcanzó la mejor puntuación. El resto de la secuencia se descarta a efectos de la salida final.

Este mecanismo complementa al token STOP: mientras que STOP permite que el modelo aprenda dónde cortar la secuencia, la extracción del mejor prefijo asegura que, aunque el modelo no haya aprendido todavía a colocar ese corte en la posición ideal, la solución reportada sea siempre la mejor sub-secuencia observada. La justificación detallada de esta política se aborda en la subsección 3.2.3, al introducir la evaluación por máximo de prefijos.

4.3. Función de evaluación

La representación del genoma fija el espacio sobre el que opera el EDA, pero el algoritmo necesita un criterio numérico que le permita ordenar a los individuos de una población según su calidad. Esa es la función de evaluación: asignar a cada candidato a solución un coste que $UMDA_{cat}$ pueda minimizar.

La elección de esta función ha sido la decisión de diseño con mayor impacto sobre el rendimiento del solver. A lo largo del desarrollo se probaron varias formulaciones, motivadas por las limitaciones observadas en cada una de ellas. La presente subsección se organiza siguiendo el orden cronológico en el que se introdujeron. Las tres primeras subsubsecciones recogen la evolución de la medida de cercanía al estado resuelto: una primera similitud basada en coincidencia directa, una segunda versión basada en la estructura de ciclos de la permutación y, finalmente, una distancia

obtenida mediante bases de datos de patrones, que es la única que el solver utiliza realmente para la selección. Las dos primeras se conservan en el código del proyecto exclusivamente con finalidad comparativa, sin intervenir en la selección. Las tres subsubsecciones restantes describen tres ajustes adicionales que se aplican sobre la medida elegida: un reward denso por fases, una evaluación por máximo de prefijos y una penalización por longitud. Estos ajustes complementan a la similitud para producir el coste final.

4.3.1. Primera versión: similitud por coincidencia directa

La función de similitud original es la introducida en la subsección 2.1.5 del estado del arte, basada en la propuesta de El-Sourani [19]. Esta similitud descompone la cercanía al estado resuelto en cuatro indicadores: la fracción de esquinas bien colocadas, la fracción de esquinas bien colocadas y orientadas, la fracción de aristas bien colocadas y la fracción de aristas bien colocadas y orientadas. Los cuatro se combinan linealmente con pesos (0,35, 0,15, 0,40, 0,10), dando lugar a una similitud global entre 0 y 1. Se eligió como primer candidato a fitness porque ya estaba formalizada para el problema y porque su definición es intuitiva.

Sin embargo, una vez integrada se observó que presenta tres limitaciones serias. La primera es que los indicadores son estrictamente binarios por pieza: cada esquina o arista contribuye o no según esté exactamente en su sitio. Un movimiento que aleja una pieza de su posición correcta produce el mismo descenso que diez movimientos que la alejan todavía más. La función no distingue entre dos configuraciones igualmente desordenadas, lo que reduce la información que el algoritmo puede aprovechar al seleccionar los mejores individuos.

La segunda limitación es la aparición de zonas planas muy amplias en el fitness. Como los cuatro indicadores son medias sobre conjuntos pequeños (7 esquinas o 9 aristas), existen pocos valores distintos que la similitud pueda tomar, y muchas configuraciones diferentes reciben la misma puntuación. Cuando una parte significativa de la población obtiene exactamente el mismo fitness, la selección pierde capacidad de discriminar y el algoritmo se estanca.

La tercera limitación, y la más problemática, es el fenómeno del engaño. Una configuración con muchas piezas en su sitio puede estar, en realidad, lejos del cubo resuelto, porque reordenar las pocas piezas restantes puede requerir secuencias largas que pasen necesariamente por estados con menos piezas correctamente colocadas. Bajo esta similitud, esos estados intermedios reciben peor puntuación que el punto de partida, lo que empuja al algoritmo a rechazarlos. El resultado es que el EDA queda atrapado en configuraciones que parecen buenas pero desde las que no puede avanzar sin empeorar temporalmente.

Por estos tres motivos, la similitud por coincidencia directa fue descartada como fitness operativo.

4.3.2. Segunda versión: similitud por longitud de ciclo

La principal carencia de la similitud original es que sus indicadores son estrictamente binarios: cada pieza está en su sitio o no. Para obtener una señal más graduada se diseñó una segunda versión que, en lugar de comprobar si cada pieza ocupa su posición correcta, mide cuán desordenada está respecto a ella.

La idea se basa en la estructura de ciclos de la permutación que describe la configuración del cubo. Cuando varias piezas están intercambiadas entre sí, forman un ciclo: por ejemplo, si la pieza A está donde debería estar la B, la B donde la C, y la C donde la A, las tres forman un ciclo de longitud 3. Una pieza que ya está en su sitio forma un ciclo de longitud 1. La puntuación de cada pieza se define como la inversa de la longitud de su ciclo: una pieza en su sitio puntúa 1, una pieza en un intercambio simple (ciclo de longitud 2) puntúa 0,5, y así sucesivamente. Cuanto más largo es el ciclo, menor es la puntuación, lo que da una señal más graduada que el simple *está o no está en su sitio*. Esta puntuación posicional se combina con la orientación de cada pieza y se agregan todas las contribuciones en una similitud global entre 0 y 1.

Frente a la similitud original, esta segunda versión supone una mejora clara. Al no ser binaria, proporciona diferencias de puntuación entre configuraciones que antes recibían el mismo valor, lo que reduce las zonas planas del fitness y permite al algoritmo distinguir mejor entre individuos de la población. La tasa de engaño también se reduce respecto a la versión original, aunque no se elimina.

Sin embargo, esta versión sigue siendo insuficiente por una razón de fondo. Por mucho que se refine, una similitud construida sobre la estructura de la permutación sigue siendo una medida indirecta que no tiene en cuenta los movimientos reales del cubo. Dos configuraciones con la misma estructura de ciclos pueden requerir un número muy distinto de movimientos para resolverse, porque los movimientos disponibles en el cubo Fuse están restringidos por su estructura mecánica. La similitud por ciclos ignora completamente esta restricción y, por tanto, puede seguir asignando buenas puntuaciones a configuraciones que en realidad están lejos de la solución.

Por este motivo, la segunda versión también se descartó como fitness operativo y la decisión fue abandonar las similitudes basadas en la permutación para adoptar una distancia que sí tuviese en cuenta los movimientos reales del cubo, tal como se describe en la siguiente subsubsección.

4.3.3. Versión final: bases de datos de patrones

Las dos versiones anteriores miden la cercanía al estado resuelto a partir de la estructura de la permutación, sin tener en cuenta los movimientos reales del cubo. En la fase final del desarrollo se sustituyó este enfoque por una medida basada en bases de datos de patrones (*pattern databases*, PDB) [20]. Esta medida tiene en cuenta los movimientos legales del cubo y proporciona distancias exactas para determinados subproblemas abstraídos. En el caso de las esquinas, la distancia se calcula sobre el subespacio completo de las 7 esquinas móviles. En el caso de las aristas, se utiliza una factorización en dos subconjuntos, por lo que la distancia resultante debe interpretarse como una aproximación de la distancia conjunta real. Esta técnica fue aplicada por Korf [1] a la resolución óptima del cubo de Rubik clásico y se traslada en este trabajo al cubo Fuse.

La primera tabla construida es la PDB de esquinas. Se exploran de forma exhaustiva todas las configuraciones alcanzables de las siete esquinas móviles, partiendo del estado resuelto y expandiendo nivel a nivel mediante todos los movimientos legales del cubo Fuse. El resultado es un diccionario que asocia a cada configuración de esquinas su distancia exacta al estado resuelto, medida en número de movimientos.

El diámetro máximo observado es $D_C = 11$, y la tabla contiene 3 674 160 estados, que corresponden al subespacio completo de las siete esquinas móviles ($7! \cdot 3^6$).

Para las aristas se presenta un problema de escala: el cubo Fuse tiene nueve aristas con dos orientaciones cada una, lo que produce un espacio de estados demasiado grande para almacenarlo en una sola tabla. La solución es dividir las nueve aristas en dos subconjuntos (cinco aristas en la primera tabla y cuatro en la segunda) y construir una PDB independiente para cada subconjunto, ignorando las posiciones y orientaciones de las aristas restantes. Esta estrategia de factorización es análoga a la empleada por Korf [1] en el cubo clásico, donde las doce aristas se dividieron en dos grupos de seis. Cada tabla resultante es mucho más pequeña y su construcción es viable, con diámetros máximos $D_5 = 12$ (piezas 0–4) y $D_4 = 10$ (piezas 5–8), y tamaños de 483 840 y 48 384 estados respectivamente.

La distancia de aristas para una configuración dada se obtiene combinando las distancias de ambas tablas. Cada distancia se normaliza a $[0, 1]$ dividiéndola por su diámetro correspondiente: $\hat{d}_5 = d_5/D_5$ y $\hat{d}_4 = d_4/D_4$. La combinación da mayor peso al subconjunto más difícil de resolver:

$$d_{\text{edge}}(s) = 0,35 \cdot \frac{\hat{d}_5 + \hat{d}_4}{2} + 0,65 \cdot \max(\hat{d}_5, \hat{d}_4).$$

La similitud de aristas se define como $\text{sim}_{\text{edge}}(s) = 1 - d_{\text{edge}}(s)$, y la similitud de esquinas de forma análoga: $\text{sim}_{\text{corner}}(s) = 1 - d_C(s)/D_C$. La similitud final basada en PDB combina ambas contribuciones:

$$\text{Sim}_{\text{PDB}}(s) = 0,45 \cdot \text{sim}_{\text{corner}}(s) + 0,55 \cdot \text{sim}_{\text{edge}}(s).$$

Los pesos están ligeramente sesgados hacia las aristas porque son más numerosas (nueve frente a siete esquinas) y porque su PDB factorizada introduce mayor incertidumbre. La función devuelve 1 solo cuando el cubo está resuelto. Las tres tablas se calculan una sola vez y se guardan en disco para reutilizarlas en todas las ejecuciones posteriores.

Esta similitud basada en PDB es la única que el solver utiliza para guiar la selección. A diferencia de las dos versiones anteriores, se basa en distancias obtenidas mediante los movimientos legales del cubo *fuse*, por lo que ofrece una señal más informada que las similitudes puramente posicionales. Sin embargo, como las distancias son valores enteros pequeños, muchas configuraciones distintas comparten el mismo valor y el fitness resultante presenta un comportamiento escalonado. La siguiente subsección aborda esta carencia introduciendo una señal complementaria que suaviza los escalones.

Para comparar el comportamiento de las funciones de evaluación consideradas, se aplicaron sobre estados obtenidos a partir de scrambles de distintas longitudes. Esta comparación no pretende medir directamente la tasa de resolución del solver, sino analizar qué tipo de señal recibe el EDA conforme el estado del cubo se aleja de la configuración resuelta.

La Figura 4.2 muestra que la similitud original pierde capacidad discriminativa muy pronto, ya que cae de forma brusca incluso con scrambles cortos. La similitud basada en ciclos mantiene una señal algo más estable, pero sigue siendo limitada. En cambio, la similitud basada en PDB presenta una disminución más progresiva, lo que justifica su elección como componente principal de la función de evaluación final.

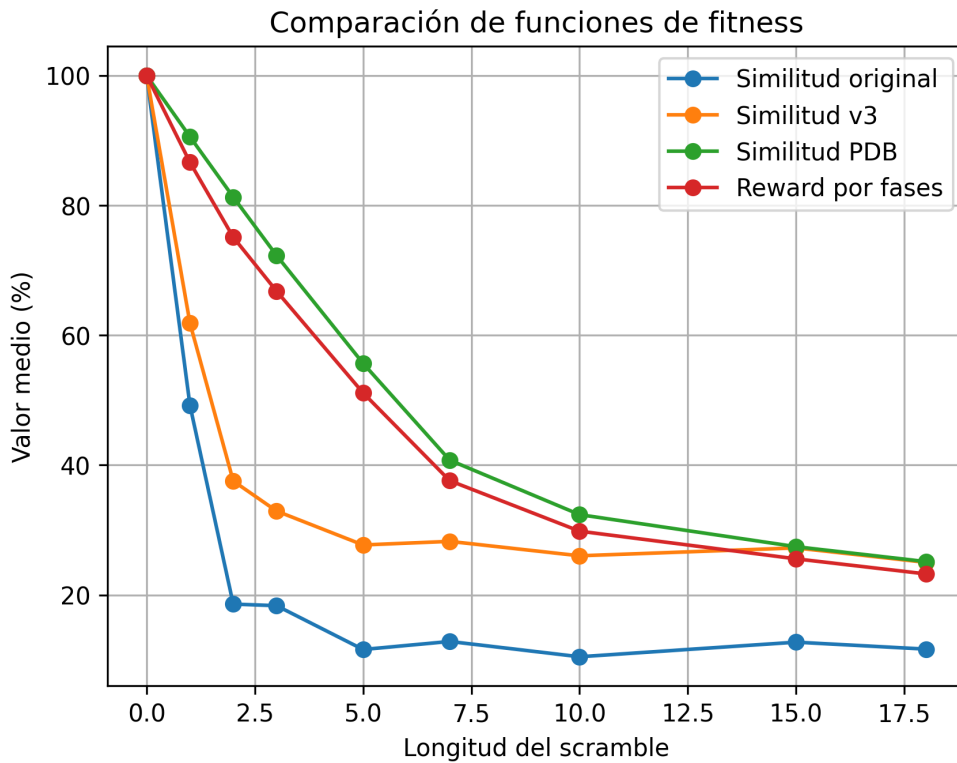


Figura 4.2: Comparación de las funciones de evaluación según la longitud del scramble.

A partir de esta comparación, se descartan las funciones basadas únicamente en coincidencia directa o en estructura de ciclos como fitness operativo del solver. La similitud basada en PDB se adopta como señal principal porque incorpora información derivada de los movimientos legales del cubo *fuse*. No obstante, al estar basada en distancias discretas, puede presentar escalones de puntuación. Por este motivo, se complementa posteriormente con un reward denso por fases.

4.3.4. Reward denso por fases

La similitud basada en PDB proporciona una distancia real al estado resuelto, pero esa distancia toma valores enteros pequeños: el diámetro de la PDB de esquinas es del orden de una docena de movimientos, por lo que muchas configuraciones distintas comparten exactamente el mismo valor. Cuando parte de la población recibe la misma puntuación, la selección no puede distinguir entre esos individuos y el algoritmo pierde capacidad de avanzar en las fases finales de la búsqueda, donde la mayoría de los individuos converge sobre distancias pequeñas.

Para suavizar estos escalones se introduce una señal complementaria que mide la fracción de piezas que están simultáneamente bien colocadas y bien orientadas. Esta señal varía cada vez que una pieza individual alcanza su estado correcto, ofreciendo un progreso más fino que la PDB. La combinación de ambas señales se realiza mediante una mezcla que mantiene la PDB como componente dominante:

$$R(s) = 0,90 \cdot \text{Sim}_{\text{PDB}}(s) + 0,10 \cdot (0,65 \cdot f_E(s) + 0,35 \cdot f_C(s)),$$

donde $f_E(s)$ y $f_C(s)$ son las fracciones de aristas y esquinas correctas, respectivamente. El factor 0,90 asignado a la PDB garantiza que la selección siga guiada por la distancia real al estado resuelto. El factor 0,10 de la señal complementaria aporta un gradiente dentro de cada escalón de la PDB sin alterar el orden general de los individuos. Dentro de la señal complementaria, el peso de las aristas (0,65) es mayor que el de las esquinas (0,35) porque las aristas involucran más piezas.

El reward está además acotado a 0,999 como máximo, salvo cuando el cubo está completamente resuelto, en cuyo caso devuelve exactamente 1,0. Esta cota reserva el valor unitario como señal exclusiva de que está resuelto, asegurando que el algoritmo pueda distinguir el cubo resuelto de cualquier configuración cercana pero no resuelta.

Sin embargo, evaluar el reward en un único punto no es suficiente. Como se explicó en la subsección 3.2.2, el genoma puede contener un buen prefijo seguido de movimientos que destruyen lo alcanzado. Si solo se evaluase el estado final, esa información se perdería. La siguiente subsubsección aborda este problema.

4.3.5. Evaluación por máximo de prefijos

Hasta este punto, la función de evaluación se ha definido sobre un único estado del cubo. Sin embargo, un individuo del EDA codifica una secuencia completa de movimientos que produce una serie de estados al ejecutarse desde el cubo mezclado. La opción más natural sería evaluar solo el estado final, pero esto presenta un problema: si el genoma contiene un buen prefijo seguido de movimientos inútiles (porque el modelo aún no ha aprendido a colocar STOP en el sitio correcto), el estado final estará destruido y el individuo recibirá mala puntuación a pesar de contener una buena solución en su interior.

La estrategia adoptada consiste en evaluar todos los estados intermedios de la secuencia y quedarse con el mejor. El solver ejecuta los movimientos uno a uno, calcula el reward $R(s)$ en cada paso y registra el punto de la secuencia que alcanza la mejor puntuación. El coste asignado al individuo se basa en ese mejor estado intermedio, no en el estado final:

$$c(\sigma) = 1 - R(s_{i^*}) + \lambda \cdot i^*,$$

donde i^* es la posición del mejor estado encontrado y λ es un coeficiente pequeño de penalización por longitud que se detalla en la siguiente subsubsección.

Cuando el cubo alcanza el estado resuelto en algún punto de la secuencia, el coste se redefine como $c(\sigma) = -100 + \lambda \cdot i$, lo que crea una separación enorme respecto al rango habitual de costes (que oscila entre 0 y 1). El efecto es que cualquier individuo que resuelva el cubo, aunque sea una sola vez durante su ejecución, obtiene un coste muy inferior al del resto de la población y queda inmediatamente favorecido por la selección. El término $\lambda \cdot i$ dentro de esta rama favorece además las soluciones más cortas entre las que resuelven el cubo.

Este mecanismo complementa al token STOP descrito en la subsección 3.2.2. Mientras que STOP permite al modelo aprender dónde terminar la secuencia, la evaluación por máximo de prefijos asegura que, aunque el modelo no haya aprendido eso todavía, los individuos que contienen buenas soluciones sean reconocidos como tales. Ambos mecanismos cooperan: el máximo de prefijos rescata individuos útiles desde

4.4. Inicialización informada del modelo probabilístico

las primeras generaciones, y con el tiempo la selección guía al modelo a colocar STOP en la posición adecuada.

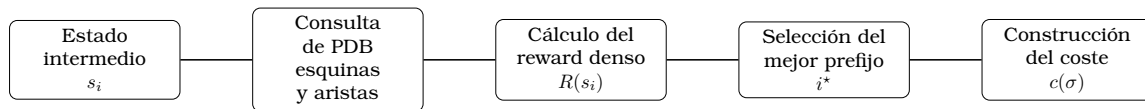
4.3.6. Penalización por longitud

El coste asociado a un individuo incluye un término $\lambda \cdot i^*$, donde i^* es la posición del mejor estado encontrado y λ es un coeficiente positivo pequeño. Este término cumple una función de desempate. Entre los individuos que resuelven el cubo, todos comparten un coste base de -100 y sin la penalización serían indistinguibles para la selección; el término $\lambda \cdot i^*$ favorece a las soluciones más cortas. Entre los individuos que no resuelven el cubo, dos prefijos pueden alcanzar el mismo reward en posiciones diferentes de la secuencia; en ese caso, la penalización favorece al prefijo más corto.

El valor elegido es $\lambda = 0,0005$. Es lo bastante pequeño como para no interferir con la señal principal del reward: con genomas de hasta treinta posiciones, el término acumulado $\lambda \cdot i^*$ no supera $0,015$, muy por debajo de las diferencias de reward habituales entre individuos. Al mismo tiempo, es suficiente para producir diferencias detectables entre soluciones de distinta longitud. Este valor se ha utilizado en todas las ejecuciones del trabajo.

Con la penalización por longitud queda completamente definida la función de evaluación del solver. En resumen, el coste final que recibe UMDA_{cat} se construye en tres pasos: primero se mide la distancia al estado resuelto mediante las PDB, combinando la tabla de esquinas con las dos tablas factorizadas de aristas; después se añade una señal de progreso fino basada en la fracción de piezas correctas para suavizar los escalones de la PDB; y finalmente se evalúa el mejor estado intermedio de la secuencia en lugar del estado final, penalizando ligeramente los prefijos largos.

La Figura 4.3 resume el proceso de construcción del coste que utiliza el solver. A partir de cada estado intermedio generado por la secuencia candidata, se calcula una señal basada en PDB, se combina con una recompensa densa asociada al progreso parcial, se selecciona el mejor prefijo observado y, finalmente, se construye el coste que guía la selección dentro de UMDA_{cat} .



$$c(\sigma) = 1 - R(s_{i^*}) + \lambda i^*, \quad c(\sigma) = -100 + \lambda i \quad \text{si el cubo se resuelve.}$$

Figura 4.3: Construcción del coste utilizado por UMDA_{cat} a partir de la señal basada en PDB, el reward denso, la evaluación por mejor prefijo y la penalización por longitud.

4.4. Inicialización informada del modelo probabilístico

La función de evaluación indica al EDA qué dirección seguir para mejorar, pero el algoritmo necesita también un punto de partida: una distribución de probabilidad inicial desde la cual generar la primera población. La elección más natural es la

distribución uniforme, que asigna la misma probabilidad a cada gen en cada posición. Sin embargo, en el problema del cubo Fuse se dispone de conocimiento del dominio que puede aprovecharse desde el inicio para evitar que las primeras generaciones se dediquen a evaluar individuos poco útiles.

Esta subsección recoge tres mecanismos mediante los cuales la inicialización se aleja de la uniforme. Los dos primeros añaden sesgos al modelo: una penalización a los movimientos consecutivos sobre la misma cara y un sesgo hacia el inverso del scramble. El tercero resuelve una particularidad de EDAspy que obliga a inyectar manualmente la primera población para que los sesgos anteriores surtan efecto.

4.4.1. Distribución uniforme y sus limitaciones

La distribución uniforme asigna a cada posición del genoma la misma probabilidad para cada uno de los diez genes posibles (10 % por gen). La primera generación muestreada bajo esta distribución es, en consecuencia, una colección de secuencias aleatorias de movimientos sin estructura aparente. Es el punto de partida más neutro y la elección adecuada cuando no se sabe nada sobre la solución.

En el problema del cubo Fuse, sin embargo, presenta dos limitaciones. La primera es que genera con frecuencia movimientos consecutivos sobre la misma cara, que la simplificación descrita en 3.2.2 colapsa o cancela. El cubo se actualiza correctamente, pero esas posiciones del genoma se desperdician sin aportar información. Como resultado, parte de la diversidad de la primera generación se gasta en variantes equivalentes de un mismo algoritmo más corto.

La segunda limitación es que las secuencias aleatorias de movimientos rara vez producen un estado cercano al resuelto. Los individuos de la primera generación obtienen puntuaciones muy similares entre sí, lo que dificulta que la selección pueda distinguir a los mejores del resto. El algoritmo necesita varias generaciones solo para empezar a generar individuos con algún avance respecto al cubo mezclado.

Las dos siguientes subsubsecciones presentan modificaciones orientadas a paliar estas limitaciones. La primera introduce una penalización a los movimientos consecutivos sobre la misma cara y se aplica en todos los reinicios. La segunda introduce un sesgo hacia el inverso del scramble y se aplica solo en el primer reinicio.

4.4.2. Penalización de cara repetida en posiciones contiguas

La primera modificación de la distribución uniforme busca reducir la aparición de movimientos consecutivos sobre la misma cara, que la simplificación post-decoding colapsa o cancela y que, por tanto, no aportan información útil. La idea es sencilla: si esos movimientos son redundantes, su probabilidad inicial debería ser menor que la del resto.

Implementar esta idea en un modelo univariado no es directo. Lo ideal sería que la probabilidad de un gen dependiese de lo que se ha muestreado en la posición anterior, pero $UMDA_{cat}$ trata cada posición de forma independiente. Como solución aproximada, se asigna a cada posición del genoma una cara penalizada distinta, siguiendo un patrón cíclico entre las tres caras del cubo (L , D , B). Los tres movimientos de la cara penalizada reciben una probabilidad baja en esa posición, mientras que los seis movimientos de las otras dos caras reciben una probabilidad mayor. El token STOP

4.4. Inicialización informada del modelo probabilístico

recibe una probabilidad pequeña y fija en todas las posiciones. La primera posición del genoma no tiene penalización, ya que no existe una posición previa con la que comparar.

Este esquema no elimina por completo los pares de movimientos consecutivos sobre la misma cara, pero reduce su frecuencia en la primera generación lo suficiente como para mejorar la calidad media de la población inicial. Los valores concretos de las probabilidades se eligen de forma que la penalización sea clara pero ningún gen quede completamente excluido, permitiendo que el algoritmo pueda explorar cualquier combinación si la selección lo requiere.

Esta tabla de frecuencias actúa como base sobre la que se construyen las demás inicializaciones. Cuando no se aplica ningún sesgo adicional, se utiliza directamente. Cuando se aplica el sesgo del inverso (siguiente subsubsección) o el sesgo hacia el mejor individuo conocido (subsección 3.2.5), esta tabla sirve como punto de partida que se modifica con el sesgo correspondiente.

4.4.3. Sesgo por el inverso del scramble

La segunda modificación aborda la baja calidad de los individuos generados aleatoriamente. Se basa en una observación directa: para cualquier scramble del cubo Fuse existe siempre una solución conocida, su inverso. Si el cubo se mezcló aplicando los movimientos $m_0, m_1, \dots, m_{k-1}$, basta con aplicar los movimientos inversos en orden contrario para deshacerlo. Invertir cada movimiento es sencillo: L se convierte en L' y viceversa, D en D' y viceversa, y las medias vueltas ($L2$, $D2$, $B2$) son su propio inverso. El resultado es una secuencia de k movimientos que siempre resuelve el cubo, aunque normalmente no es la solución más corta posible.

La inicialización aprovecha esta solución conocida sesgando la distribución inicial hacia ella. En cada posición del genoma que corresponde a un movimiento del inverso, ese movimiento recibe una probabilidad alta mientras que los demás se reparten el resto. En la posición justo después del último movimiento del inverso, el token STOP recibe la mayor parte de la probabilidad, indicando al modelo que ahí debería terminar la secuencia. Para las posiciones restantes, si el genoma es más largo que el inverso, se utiliza la tabla de penalización de cara repetida descrita en la subsubsección anterior. Además, si dos posiciones consecutivas del inverso involucran la misma cara, se reduce el sesgo en la segunda posición para evitar que la simplificación las colapse y se desperdicie longitud útil.

El sesgo es deliberadamente blando: el movimiento del inverso recibe una probabilidad mayoritaria pero no absoluta, dejando margen para que el algoritmo explore alternativas. Esto es importante porque el inverso no suele ser la solución más eficiente. En muchos casos el cubo admite soluciones más cortas, y el papel del sesgo es simplemente dar al modelo un buen punto de partida desde el que la evolución pueda mejorar. Por este motivo, esta inicialización se aplica únicamente en el primer reinicio de cada ejecución. En reinicios posteriores, el modelo se inicializa a partir del mejor individuo encontrado hasta el momento, que aporta información más útil que el inverso original.

4.4.4. Generación coherente de la primera población

Los sesgos descritos en las dos subsubsecciones anteriores definen la distribución de probabilidad que el solver quiere utilizar como punto de partida. Sin embargo, la implementación de $UMDA_{cat}$ en EDAspy presenta una particularidad: la tabla de frecuencias que se le proporciona no se utiliza para generar la primera población. Si no se interviene, la generación inicial se crea con un procedimiento por defecto que ignora los sesgos introducidos, y estos solo empezarían a actuar a partir de la segunda generación.

Para que los sesgos tengan efecto desde el primer momento, el solver genera la primera población de forma manual a partir de la tabla de frecuencias configurada y se la proporciona directamente al algoritmo como población inicial. A partir de la segunda generación, $UMDA_{cat}$ recupera el control del muestreo y aplica su lógica habitual. La intervención manual afecta únicamente a la generación cero.

Con esto queda completamente definida la inicialización del modelo probabilístico. En resumen, sobre la distribución uniforme se aplican dos capas de sesgo (penalización de cara repetida y, en el primer reinicio, sesgo hacia el inverso del scramble) y se asegura que la primera población refleje efectivamente esos sesgos mediante la inyección manual descrita.

4.5. Estrategia de exploración con reinicios

Las subsecciones anteriores definen los componentes del solver para una única ejecución de $UMDA_{cat}$: el genoma, la función de evaluación y la inicialización. Sin embargo, una sola ejecución no suele ser suficiente para resolver un scramble del cubo Fuse, por dos razones. La primera es que la longitud del genoma es fija y debe elegirse antes de ejecutar el algoritmo: si es demasiado corta, no cabe la solución; si es demasiado larga, el espacio de búsqueda crece mucho y la convergencia se ralentiza. La segunda es que, incluso con una longitud adecuada, el algoritmo puede quedarse atrapado en una zona del espacio de búsqueda sin encontrar mejoras.

Para abordar ambos problemas, el solver organiza su ejecución como una secuencia de reinicios con longitud de genoma creciente. Cada reinicio puede aprovechar la información de los anteriores y dispone de mecanismos para escapar del estancamiento. Esta subsección describe los cuatro componentes de esta estrategia: la planificación de longitudes crecientes (3.2.5.1), el sesgo hacia el mejor individuo conocido entre reinicios (3.2.5.2), la mezcla de frecuencias cuando el algoritmo se estanca dentro de un reinicio (3.2.5.3) y un registro auxiliar de las mejores soluciones observadas (3.2.5.4).

4.5.1. Longitud creciente del genoma

La longitud del genoma L determina la secuencia de movimientos más larga que el algoritmo puede explorar. Si L es menor que la longitud de cualquier solución posible, el solver no puede resolver el cubo. Si L es excesivamente grande, el espacio de búsqueda crece y el algoritmo pierde eficiencia.

Como no se conoce de antemano la longitud de la solución para un scramble concreto, la estrategia adoptada es empezar con un genoma corto e ir aumentándolo en cada

4.5. Estrategia de exploración con reinicios

reinicio. Sea ℓ_0 la longitud inicial, Δ el incremento por reinicio y $\ell_{\text{máx}}$ la longitud máxima; el reinicio de índice r utiliza una longitud:

$$L_r = \min(\ell_0 + r \cdot \Delta, \ell_{\text{máx}}).$$

Por defecto, ℓ_0 se fija como la longitud del scramble, ya que el inverso del scramble es siempre una solución válida de esa longitud y garantiza que el espacio de búsqueda inicial contenga al menos una solución. El valor $\ell_{\text{máx}}$ se establece como tres veces la longitud del scramble, y el incremento Δ es de una o dos posiciones por reinicio.

Empezar con genomas cortos tiene dos ventajas. Por un lado, un espacio de búsqueda más pequeño permite al algoritmo converger más rápido, y solo se amplía si no se ha encontrado solución. Por otro lado, al crecer gradualmente, el conocimiento adquirido en un reinicio puede transferirse al siguiente de forma más efectiva, como se describe en la subsubsección 3.2.5.2. La secuencia de reinicios se recorre en orden hasta que el solver resuelve el cubo o hasta que se agota el número máximo de reinicios, en cuyo caso se devuelve el mejor individuo encontrado durante toda la ejecución.

4.5.2. Reinicios cálidos: sesgo hacia el mejor individuo conocido

Cuando un reinicio termina sin resolver el cubo, el mejor individuo encontrado durante su ejecución contiene información valiosa. Descartar esa información e inicializar el reinicio siguiente desde cero obligaría al algoritmo a redescubrir secuencias que ya había identificado como prometedoras.

Para evitarlo, a partir del segundo reinicio la distribución inicial se sesga hacia el mejor individuo acumulado hasta ese momento. Concretamente, cada posición del genoma conserva un 65 % de la distribución base (la penalización de cara repetida) y concentra el 35 % restante en el gen que ocupaba esa posición en el mejor individuo conocido. El sesgo resultante es más suave que el del inverso del scramble (50 %), porque el mejor individuo no es necesariamente una buena solución: puede estar atrapado en un óptimo local. Un sesgo demasiado fuerte hacia un individuo mediocre perjudicaría la búsqueda, mientras que uno demasiado débil desperdiciaría la información acumulada. Si el genoma del nuevo reinicio es más largo que el del individuo heredado, las posiciones sobrantes se sesgan hacia STOP, indicando que la secuencia probablemente debería terminar ahí.

En el primer reinicio, donde aún no existe un individuo previo, se utiliza en su lugar el sesgo por el inverso del scramble descrito en la subsubsección 3.2.4.3.

4.5.3. Mezcla de frecuencias por estancamiento dentro de un reinicio

Dentro de una misma ejecución de UMDA_{cat}, el modelo puede estancarse: la distribución converge sobre una región del espacio de búsqueda y las generaciones sucesivas producen individuos cada vez más parecidos sin mejorar el fitness. Para detectar esta situación, el solver cuenta las evaluaciones consecutivas sin mejora. Cuando se acumulan 15 generaciones completas sin progreso, se activa un mecanismo de recuperación.

La acción consiste en mezclar la distribución que el modelo ha aprendido hasta el momento con la distribución inicial del reinicio. La mezcla conserva solo un 30 % del modelo aprendido y toma un 70 % de la distribución inicial. Este reinicio es intencionadamente agresivo: si el modelo se ha estancado, lo que ha aprendido no está

siendo útil, y es preferible volver en gran parte al punto de partida para reabrir la exploración. El 30 % retenido evita perder por completo las tendencias que sí pudieran ser valiosas. Tras la mezcla, el contador de estancamiento se reinicia a cero y el mecanismo puede activarse de nuevo si el algoritmo vuelve a estancarse.

4.5.4. Registro de élites

Además de los mecanismos anteriores, el solver mantiene una lista con las 24 mejores soluciones observadas a lo largo de toda la ejecución, sin duplicados. Este registro no interviene en la selección ni en la actualización del modelo: no es un mecanismo de selección elitista. Su función es exclusivamente de diagnóstico, permitiendo inspeccionar al final de la ejecución las mejores secuencias descubiertas y analizar la diversidad de soluciones exploradas.

Con el registro de élites queda descrita la estrategia completa de reinicios. En resumen, el solver combina cuatro mecanismos: longitudes de genoma crecientes para adaptar la capacidad del modelo, sesgo hacia el mejor individuo conocido para transferir información entre reinicios, mezcla de frecuencias para recuperarse del estancamiento dentro de un reinicio, y un registro de las mejores soluciones para el análisis posterior. Estos mecanismos, junto con la función de evaluación y la inicialización informada descritas en las subsecciones anteriores, constituyen el solver completo.

4.6. Fortalezas del diseño

Una de las propiedades más destacables del solver es que no depende de estrategias de resolución diseñadas por expertos. A diferencia de métodos clásicos como CFOP [21], Roux [22] o el algoritmo de dos fases de Kociemba [23], que descomponen la resolución en etapas predefinidas con secuencias memorizadas, el EDA desarrollado explora directamente el espacio de secuencias de movimientos sin imponer una estructura fija. Ningún paso del solver codifica una estrategia humana del tipo *resolver primero la capa inferior y luego orientar las esquinas*. Esto hace que el enfoque sea potencialmente transferible a otros puzzles combinatorios con un alfabeto de movimientos distinto, redefiniendo la representación del estado, las PDB [20] y el alfabeto.

Esto no significa que el solver ignore el dominio del problema. Las bases de datos de patrones proporcionan una distancia real construida desde los movimientos del cubo. El inverso del scramble aporta una solución conocida como punto de partida. La penalización de cara repetida aprovecha una propiedad del grupo de movimientos. Estas formas de conocimiento describen propiedades del problema sin prescribir cómo resolverlo, lo que permite al EDA descubrir soluciones que una estrategia manual podría no encontrar.

La arquitectura incluye además varios mecanismos de robustez que se complementan entre sí: el token STOP para aprender la longitud de la solución, la evaluación por máximo de prefijos para reconocer buenas soluciones dentro de genomas ruidosos, los reinicios cálidos para transferir conocimiento entre ejecuciones, la mezcla de frecuencias para escapar del estancamiento y las longitudes crecientes del genoma para adaptarse a la profundidad del scramble. Cada uno aborda una vulnerabilidad concreta del EDA univariado, y juntos producen un solver más robusto que cualquier ejecución aislada del algoritmo.

4.7. Limitaciones del diseño

El solver no busca soluciones óptimas. Las secuencias encontradas son, en general, más largas que la solución mínima, ya que la evaluación por máximo de prefijos favorece el primer buen prefijo encontrado, no el más corto posible. La penalización por longitud introduce una preferencia suave por soluciones cortas, pero no garantiza optimalidad. Además, la tasa de éxito del solver disminuye a medida que el scramble es más largo: el espacio de búsqueda crece con la longitud del genoma y es cada vez más difícil que un individuo aleatorio contenga un prefijo útil.

La principal limitación estructural es el modelo univariado. Al tratar cada posición del genoma de forma independiente, UMDA_{cat} no puede capturar dependencias entre posiciones adyacentes que claramente existen en el problema, como la conveniencia de no repetir cara en movimientos consecutivos. La simplificación post-decoding y la penalización de cara repetida mitigan esta carencia pero no la eliminan. Un modelo que capturase esas dependencias podría ser más eficaz, aunque a costa de un mayor coste computacional. El modelo univariado también es propenso a converger prematuramente cuando la población es pequeña; la mezcla de frecuencias aborda este riesgo parcialmente pero no lo elimina por completo.

El coste computacional es también una consideración relevante. Cada evaluación requiere ejecutar la secuencia de movimientos completa sobre el estado del cubo y consultar las PDB en cada estado intermedio para la evaluación por máximo de prefijos, lo que hace que el coste por individuo crezca con la longitud del genoma. Multiplicado por el tamaño de la población, el número de generaciones y los reinicios, el volumen total de consultas puede ser elevado. La construcción de las PDB es un coste inicial que se realiza una sola vez, pero la PDB factorizada de aristas pierde cierta precisión al rastrear solo subconjuntos de piezas, y su combinación es una aproximación de la distancia conjunta real.

Con las fortalezas y limitaciones identificadas queda cerrada la descripción del diseño del algoritmo de estimación de distribuciones para el cubo Fuse. El capítulo 4 presentará los resultados experimentales y evaluará en la práctica el alcance de las fortalezas y la gravedad de las limitaciones aquí señaladas.

Capítulo 5

Resultados y conclusiones

En este capítulo se presentan las pruebas realizadas para validar la implementación del cubo *fuse* y analizar el comportamiento del solver basado en $UMDA_{cat}$. El objetivo de esta evaluación no es demostrar que el algoritmo obtenga soluciones óptimas, sino estudiar hasta qué punto el enfoque propuesto es capaz de resolver el cubo o aproximarse al estado resuelto bajo distintas condiciones de dificultad.

La evaluación se divide en varias partes. En primer lugar, se comprueba que la infraestructura del cubo funciona correctamente mediante pruebas sobre movimientos, inversos, estado resuelto y aplicación de scrambles. Después, se estudia el rendimiento del solver sobre scrambles de distintas longitudes, atendiendo a métricas como la tasa de resolución, la longitud de la solución encontrada, el tiempo de ejecución y el intervalo de confianza de Wilson. Por último, se verifica de forma independiente que las soluciones marcadas como resueltas realmente llevan el cubo al estado resuelto.

5.1. Resultados

En esta sección se presentan los resultados obtenidos durante la evaluación del solver. Primero se describe la metodología experimental utilizada, después se resume la validación funcional del cubo y, por último, se analizan los resultados del benchmark realizado sobre scrambles de distintas profundidades.

La comparación entre funciones de evaluación se ha tratado en el capítulo anterior, donde se justificó la elección de la señal basada en PDB y del reward denso por fases. En este capítulo se utiliza ya la función de evaluación final para analizar el rendimiento del solver.

5.1.1. Metodología experimental

Para evaluar el sistema se realizaron dos tipos de pruebas. El primer grupo corresponde a pruebas funcionales, cuyo objetivo es comprobar que la infraestructura del cubo *fuse* está correctamente implementada. Estas pruebas verifican propiedades básicas del modelo, como la existencia del estado resuelto, la correcta aplicación de movimientos, la reversibilidad mediante movimientos inversos y la validez de los estados generados.

El segundo grupo corresponde a pruebas experimentales del solver. En este caso, se generó un banco de scrambles de distintas profundidades y se ejecutó el algoritmo sobre cada uno de ellos. La profundidad del scramble se utiliza como una aproximación de la dificultad del problema, ya que scrambles más largos suelen generar configuraciones más alejadas del estado resuelto. No obstante, esta relación no es exacta, porque una secuencia larga puede contener cancelaciones o conducir a un estado que admita una solución más corta que la propia secuencia utilizada para generarlo.

En cada ejecución, el solver parte del estado mezclado y trata de encontrar una secuencia de movimientos que lleve el cubo al estado resuelto. Si durante la evaluación se alcanza el estado resuelto, la ejecución se considera satisfactoria. En caso contrario, se registra como no resuelta dentro del presupuesto de búsqueda utilizado.

Las profundidades evaluadas fueron 5, 10, 16 y 18 movimientos. Para cada profundidad se utilizaron 50 scrambles, manteniendo la misma configuración del solver en todos los casos. De esta forma, los resultados pueden compararse de manera homogénea entre profundidades.

Las métricas principales utilizadas en los experimentos son las siguientes:

- **Tasa de resolución:** porcentaje de ejecuciones en las que el solver alcanza el estado resuelto.
- **Longitud de la solución:** número de movimientos de la secuencia devuelta por el solver en los casos resueltos.
- **Tiempo de ejecución:** tiempo necesario para completar la búsqueda en cada caso.
- **Intervalo de confianza:** intervalo de Wilson al 95 % para estimar la variabilidad de la tasa de resolución.

Además, los experimentos se interpretan teniendo en cuenta la naturaleza probabilística del algoritmo. Al tratarse de un EDA, el proceso depende del muestreo de poblaciones y de la actualización del modelo probabilístico. Por este motivo, el benchmark se ejecuta con semillas controladas, de forma que los resultados queden asociados a una configuración concreta del código y puedan reproducirse posteriormente.

5.1.2. Validación funcional del cubo

Antes de evaluar el solver, se comprobó que la implementación del cubo *fuse* funcionase correctamente. Esta validación es necesaria porque cualquier error en la definición de los movimientos, en la orientación de las piezas o en la representación del estado afectaría directamente a los resultados del algoritmo.

Las pruebas realizadas se centraron en propiedades básicas del modelo:

- comprobar que el estado inicial coincide con el estado resuelto;
- verificar que aplicar un movimiento y después su inverso devuelve el cubo al estado anterior;
- comprobar que aplicar cuatro veces un mismo giro de 90° devuelve el cubo al estado original;

Resultados y conclusiones

- verificar que los movimientos dobles son equivalentes a aplicar dos veces el movimiento simple correspondiente;
- comprobar que un scramble seguido de su inverso devuelve el cubo al estado resuelto;
- verificar que los estados generados mantienen la estructura válida del cubo.

Los resultados de estas pruebas fueron satisfactorios. Por tanto, se considera que la infraestructura del cubo es adecuada para ser utilizada por el solver. Esta validación no demuestra que el algoritmo de búsqueda sea eficaz, pero sí permite asegurar que las secuencias de movimientos evaluadas por el EDA se aplican sobre un modelo coherente del cubo *fuse*.

5.1.3. Resultados del solver

Una vez validada la infraestructura del cubo y definida la función de evaluación final, se ejecutó un benchmark para analizar el comportamiento del solver sobre scrambles de distintas profundidades. El objetivo de esta prueba no es comprobar si el algoritmo encuentra soluciones óptimas, sino estudiar hasta qué profundidad es capaz de resolver el cubo *fuse* utilizando el diseño propuesto.

La variante evaluada corresponde al solver basado en $UMDA_{cat}$ con inicialización informada. Se utilizó una población de 500 individuos, un máximo de 200 generaciones por reinicio, parada por estancamiento tras 30 generaciones sin mejora, una fracción de selección $\alpha = 0,5$, una penalización por longitud $\lambda = 0,0005$, un incremento de longitud de genoma igual a 2 y un máximo de 10 reinicios.

La configuración evaluada incorpora conocimiento del dominio en la inicialización del modelo probabilístico. En concreto, se utiliza información del scramble para favorecer secuencias compatibles con una posible resolución. Esta decisión es coherente con el planteamiento del trabajo, ya que el objetivo no es realizar una búsqueda completamente ciega, sino estudiar si un EDA puede aprovechar una señal inicial informada y guiarla mediante selección probabilística.

La Tabla 5.1 resume los resultados obtenidos para cada profundidad. Se muestra el número total de ejecuciones, el número de casos resueltos, la tasa de éxito, el intervalo de confianza de Wilson al 95 %, la longitud media de las soluciones encontradas y el tiempo medio de ejecución.

Cuadro 5.1: Resultados del solver por profundidad de scramble.

Profundidad	Ejecuciones	Resueltas	Tasa de éxito	IC Wilson 95 %	Longitud media	Tiempo medio (s)
5	50	50	100,0 %	[93, 100]	5,0	0,8
10	50	50	100,0 %	[93, 100]	10,0	1,6
16	50	36	72,0 %	[58, 82]	16,0	11,2
18	50	21	42,0 %	[29, 56]	18,0	16,5

La Figura 5.1 representa gráficamente la evolución de la tasa de resolución conforme aumenta la profundidad del scramble.

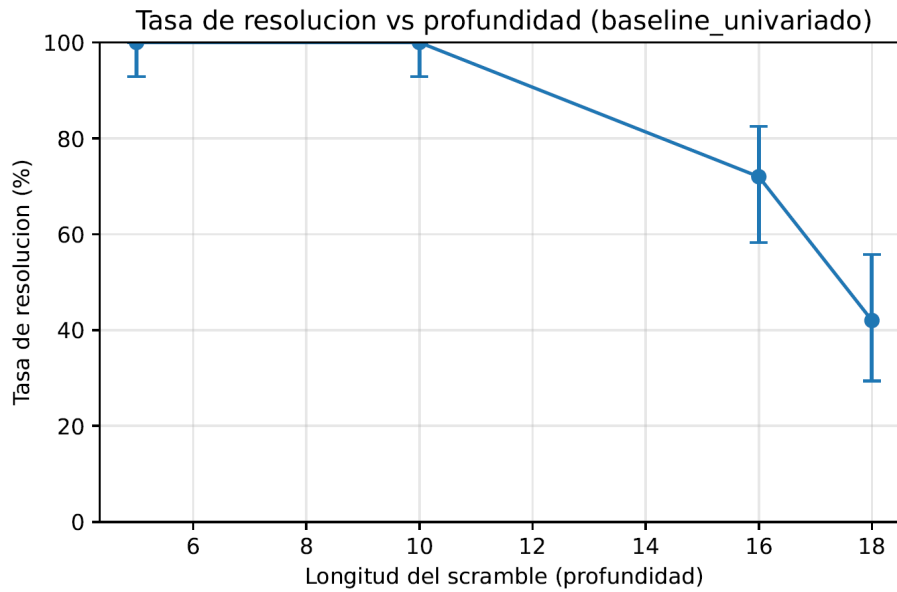


Figura 5.1: Tasa de resolución del solver según la longitud del scramble.

Los resultados muestran que el solver resuelve todos los casos de profundidad 5 y 10. En estas profundidades, la combinación de inicialización informada, función de evaluación basada en PDB, token *STOP*, evaluación por mejor prefijo y reinicios permite encontrar soluciones válidas de forma consistente.

A partir de profundidad 16 se observa una caída clara en la tasa de resolución. El solver resuelve 36 de los 50 casos, lo que equivale a una tasa de éxito del 72%. En profundidad 18, la tasa desciende hasta el 42%, con 21 casos resueltos de 50. Esta disminución indica que, aunque la inicialización informada proporciona una guía útil, el modelo tiene más dificultades para conservar y explotar la información necesaria cuando aumenta la longitud de la secuencia.

El aumento de la dificultad también se refleja en el tiempo medio de ejecución. Para profundidad 5, el tiempo medio es de 0,8 segundos, mientras que para profundidad 10 asciende a 1,6 segundos. En profundidad 16 el tiempo medio aumenta hasta 11,2 segundos y en profundidad 18 alcanza 16,5 segundos. Esto muestra que el solver necesita más esfuerzo computacional conforme aumenta la profundidad del scramble.

La longitud media de las soluciones encontradas coincide con la profundidad del scramble. Esto se debe a que, en los casos resueltos, el solver aprovecha la información proporcionada por la inicialización informada y tiende a recuperar secuencias equivalentes al inverso del scramble. Este resultado no se interpreta como una limitación del enfoque, sino como una consecuencia esperable del uso de conocimiento del dominio. En problemas combinatorios con espacios de búsqueda amplios, proporcionar una guía inicial puede ser necesario para que una metaheurística probabilística encuentre soluciones en un tiempo razonable.

Por tanto, los resultados no muestran únicamente que exista una solución conocida a partir del scramble, sino que $UMDA_{cat}$ es capaz de conservar, seleccionar y explotar esa información dentro de una población de secuencias candidatas. El algoritmo no

se limita a aplicar directamente una regla fija, sino que trabaja con poblaciones generadas probabilísticamente, evalúa sus individuos, selecciona los mejores y actualiza el modelo en función de los resultados obtenidos.

5.1.4. Verificación independiente de las soluciones

Como comprobación adicional, se verificaron de forma independiente todas las soluciones marcadas como resueltas. Esta verificación se realizó mediante un script separado del benchmark principal, con el objetivo de evitar falsos positivos producidos por un posible error en el registro de resultados.

Para cada solución marcada como resuelta, el procedimiento reconstruye el caso desde el estado resuelto, aplica primero el scramble y después la solución devuelta por el solver. A continuación, comprueba si el estado final coincide exactamente con el estado resuelto. Esta comprobación se realiza sobre las cuatro componentes internas de la representación del cubo: permutación de esquinas, orientación de esquinas, permutación de aristas y orientación de aristas.

En total se comprobaron 157 soluciones y no se detectó ningún error. Por tanto, todas las soluciones reportadas como resueltas pueden considerarse válidas.

5.1.5. Discusión de resultados

Los resultados obtenidos muestran que el solver es eficaz en scrambles cortos y medios. Para profundidades 5 y 10 se alcanza una tasa de resolución del 100 %, lo que indica que la combinación de inicialización informada, función de evaluación basada en PDB, token *STOP*, evaluación por mejor prefijo y reinicios permite resolver estos casos de forma consistente.

A partir de profundidad 16 aparece una caída clara en el rendimiento. Aunque el solver todavía resuelve el 72 % de los casos, la tasa de éxito deja de ser total. En profundidad 18, el porcentaje de resolución baja hasta el 42 %, lo que permite identificar una frontera práctica de dificultad para la configuración evaluada.

Esta caída es coherente con la naturaleza del modelo utilizado. $UMDA_{cat}$ aprende distribuciones marginales por posición del genoma, pero no modela explícitamente dependencias entre movimientos consecutivos. Por tanto, cuando la longitud de la secuencia aumenta, resulta más difícil conservar una estructura completa de resolución dentro del proceso de muestreo.

También debe destacarse que el uso de inicialización informada no debe interpretarse como una debilidad del experimento. En un problema combinatorio de este tipo, proporcionar conocimiento del dominio es una decisión razonable para evitar una búsqueda prácticamente aleatoria. Los resultados muestran que el EDA es capaz de aprovechar esa información, conservarla parcialmente en la población y transformarla en soluciones válidas mediante el proceso de selección y actualización probabilística.

En conjunto, el comportamiento observado confirma que el enfoque propuesto es viable para scrambles de dificultad baja y media, pero presenta limitaciones cuando la profundidad aumenta. Estas limitaciones no invalidan el diseño, sino que delimitan su alcance y señalan posibles mejoras futuras.

5.2. Conclusiones personales

Al terminar este trabajo, considero que se han cumplido los objetivos principales que se plantearon al inicio del proyecto. Se ha conseguido modelar una variante del cubo de Rubik con restricciones, definir sus piezas móviles, representar sus estados y construir un conjunto de movimientos válidos para poder trabajar con él de forma computacional. Además, se ha desarrollado un solver basado en un Algoritmo de Estimación de Distribuciones capaz de resolver scrambles de distintas profundidades.

Una de las conclusiones más importantes que extraigo es que la representación del problema es una parte fundamental del trabajo. Al principio puede parecer que lo más importante es el algoritmo de búsqueda, pero durante el desarrollo he comprobado que, si el cubo no está bien representado, ningún solver puede funcionar correctamente. Definir bien las esquinas, las aristas, las orientaciones, los movimientos y el estado resuelto ha sido necesario para poder construir después el resto del sistema.

También he aprendido que un EDA no puede aplicarse de forma completamente genérica a un problema como este y esperar que encuentre soluciones sin ningún tipo de ayuda. El espacio de búsqueda es demasiado grande y las soluciones dependen mucho del orden de los movimientos. Por eso, ha sido necesario incorporar conocimiento del dominio, como la inicialización informada, el uso de bases de datos de patrones, el token *STOP*, la evaluación por mejor prefijo y la penalización por longitud. Estas decisiones han sido importantes para que el algoritmo no busque de manera casi aleatoria.

Otro aspecto relevante ha sido la evolución de la función de evaluación. Durante el trabajo se probaron varias formas de medir la cercanía al estado resuelto. Las primeras funciones, basadas en coincidencia directa o en ciclos, eran más sencillas, pero no daban una señal suficientemente útil para guiar al solver. La función basada en PDB y el reward denso por fases permitieron obtener una evaluación más informada del estado del cubo. Esto me ha ayudado a entender que, en problemas de optimización, la función de evaluación puede ser tan importante como el propio algoritmo.

Los resultados obtenidos muestran que el enfoque propuesto funciona bien en scrambles cortos y medios. El solver consigue resolver todos los casos de profundidad 5 y 10, y mantiene una tasa de éxito razonable en profundidad 16. Sin embargo, en profundidad 18 la tasa de resolución baja de forma clara. Esto indica que el método tiene una frontera práctica de dificultad. Para mí, esta caída no invalida el trabajo, sino que muestra de forma realista hasta dónde llega el modelo implementado.

La principal limitación que he observado está relacionada con el uso de UMDA_{cat} como modelo univariado. Este modelo aprende probabilidades por posición del genoma, pero no aprende directamente relaciones entre varios movimientos consecutivos. En un problema como el cubo, donde el orden de los movimientos es muy importante, esta limitación se nota especialmente cuando aumenta la profundidad del scramble.

En resumen, este trabajo me ha permitido entender mejor cómo se puede aplicar conocimiento a un problema combinatorio con restricciones. He comprobado que no basta con implementar un algoritmo, sino que hay que adaptar la representación, la función de evaluación y la estrategia de búsqueda al problema concreto. Aunque el solver no es óptimo y tiene limitaciones en scrambles largos, considero que el resultado es positivo, ya que demuestra que un EDA puede utilizarse de forma útil

para resolver una variante restringida del cubo de Rubik cuando se combina con conocimiento del dominio.

5.3. Trabajo futuro

Aunque el solver desarrollado cumple los objetivos principales del trabajo, existen varias líneas que podrían explorarse para mejorar su rendimiento en scrambles más largos. Estas mejoras no se han incluido como parte central de la versión final evaluada en esta memoria, pero permiten identificar hacia dónde podría evolucionar el proyecto.

Una primera línea de trabajo sería estudiar modelos EDA capaces de capturar dependencias entre movimientos consecutivos. En la versión principal se utiliza $UMDA_{cat}$, que modela cada posición del genoma de forma independiente. Esta decisión simplifica el algoritmo y reduce el coste computacional, pero también limita su capacidad para aprender patrones formados por varios movimientos seguidos. En un problema como el cubo *fuse*, donde el orden de los movimientos es fundamental, esta limitación se vuelve más importante cuando aumenta la profundidad del scramble.

Durante el desarrollo se realizaron pruebas exploratorias, con ayuda de una herramienta de asistencia al código, para generar variantes alternativas del EDA. Estas versiones no se consideran parte principal del trabajo, ya que requerirían una revisión manual más profunda, una integración más limpia y una validación completa antes de incorporarse formalmente al solver. Aun así, sus resultados son útiles para orientar el trabajo futuro.

Entre las variantes exploradas se encuentran versiones sin token *STOP*, modelos con transiciones tipo Markov y una versión con anclaje progresivo de prefijos. La idea principal de estas variantes es permitir que el modelo no solo aprenda qué movimiento es más probable en cada posición, sino también qué movimientos tienden a aparecer después de otros. Esto introduce una forma sencilla de dependencia entre posiciones consecutivas del genoma, manteniendo el planteamiento general de un EDA basado en muestreo, selección y actualización probabilística.

Los resultados preliminares de estas variantes fueron prometedores. En particular, la versión denominada *markov_anchor* alcanzó tasas de resolución muy superiores a la línea base en scrambles largos: obtuvo un 100% de éxito en profundidad 15 y un 92% en profundidad 19. Sin embargo, en profundidad 25 la tasa de éxito fue todavía baja, alrededor del 20%, por lo que el problema sigue sin estar completamente resuelto para scrambles más profundos. Estos resultados sugieren que incorporar dependencias entre movimientos puede mejorar mucho el comportamiento del solver, pero también muestran que la dificultad crece de forma clara al aumentar la longitud del scramble.

Otra línea futura sería profundizar en la estrategia de anclaje de prefijos. Esta idea consiste en fijar una parte de la mejor secuencia encontrada cuando el algoritmo se estanca y relanzar el EDA sobre el subproblema restante. De esta forma, el solver no intenta resolver toda la secuencia de una sola vez, sino que construye la solución por fases. Esta estrategia parece especialmente interesante para scrambles largos, ya que reduce la dificultad efectiva de cada etapa de búsqueda.

También sería interesante estudiar el efecto de aumentar el número de fases de an-

claje, reiniciar el modelo probabilístico entre fases o utilizar poblaciones mayores en las fases finales. Estas modificaciones podrían ayudar a mejorar los resultados en profundidades superiores, especialmente en casos como profundidad 25, donde las variantes exploratorias todavía presentan una tasa de éxito limitada.

Una mejora adicional sería comparar de forma más sistemática distintas familias de EDAs. Por ejemplo, podrían probarse modelos bivariados o multivariados que aprendan dependencias más ricas entre posiciones del genoma. Estos modelos tendrían un coste computacional mayor, pero podrían adaptarse mejor a la naturaleza secuencial del problema.

También queda abierta la posibilidad de combinar el EDA con técnicas auxiliares sin abandonar el enfoque principal del trabajo. Por ejemplo, se podrían estudiar mecanismos de búsqueda local ligera sobre los mejores individuos, estrategias de reparación de secuencias o nuevas formas de construir la función de evaluación. Estas mejoras deberían evaluarse con cuidado para mantener claro qué parte del comportamiento corresponde al EDA y qué parte procede de otros mecanismos añadidos.

Por último, el modelo podría extenderse a otras variantes *bandaged* del cubo de Rubik. Para ello sería necesario redefinir el bloque rígido, el conjunto de piezas móviles, los movimientos legales y las bases de datos de patrones. Esta extensión permitiría comprobar si la arquitectura propuesta es realmente transferible a otros rompecabezas con restricciones mecánicas.

En resumen, el trabajo futuro más prometedor consiste en avanzar desde un modelo univariado hacia modelos que aprendan dependencias entre movimientos. Las pruebas exploratorias asistidas por herramientas de generación de código indican que esta dirección puede mejorar mucho la tasa de resolución en scrambles largos, aunque todavía sería necesario revisar, depurar y validar formalmente estas variantes antes de considerarlas parte definitiva del solver.

Capítulo 6

Impacto del trabajo

6.1. Impacto general

El presente trabajo se sitúa en un ámbito poco explorado dentro de la optimización combinatoria: la aplicación de Algoritmos de Estimación de Distribuciones a un rompecabezas con restricciones de movimiento, como es el cubo *fuse*. El cubo de Rubik $3 \times 3 \times 3$ convencional cuenta con una literatura amplia, desde métodos óptimos basados en bases de datos de patrones hasta enfoques más recientes basados en aprendizaje por refuerzo. Sin embargo, las variantes *bandaged* o *fuse* han recibido mucha menos atención desde el punto de vista algorítmico.

Por este motivo, el principal impacto académico del proyecto no consiste en superar a los solvers óptimos del cubo clásico, ya que ese no es el objetivo del trabajo. Su aportación está en estudiar si un EDA puede aplicarse a un problema con un espacio de movimientos reducido, restricciones mecánicas y caminos de resolución menos directos que los del cubo estándar.

Desde el punto de vista técnico, el solver se ha diseñado de forma que varias de sus ideas puedan reutilizarse en otros problemas similares. La representación de un individuo como una secuencia de movimientos, el uso del token *STOP*, la evaluación por mejor prefijo, la penalización por longitud y la estrategia de reinicios no dependen exclusivamente del cubo *fuse*. Si se redefine la representación del estado, las bases de datos de patrones y el alfabeto de movimientos, el mismo esquema podría adaptarse a otros rompecabezas o problemas combinatorios definidos mediante operadores discretos.

Otro aspecto relevante del trabajo es la reproducibilidad. Los experimentos se realizan a partir de scrambles generados de forma controlada y con parámetros definidos. Además, las soluciones obtenidas se verifican reconstruyendo el estado final del cubo y comprobando si coincide exactamente con el estado resuelto. Esto reduce el riesgo de aceptar como válida una solución incorrecta y permite analizar los resultados con mayor confianza.

El trabajo también tiene un valor didáctico, ya que documenta todo el proceso de diseño del solver. No se presenta únicamente la versión final, sino también la evolución de la función de evaluación: primero mediante coincidencia directa, después mediante ciclos de permutación y finalmente mediante bases de datos de patrones.

Esta evolución permite justificar por qué unas funciones fueron descartadas y por qué la versión final proporciona una señal más útil para guiar la búsqueda.

Por último, algunas decisiones de diseño se tomaron teniendo en cuenta el coste computacional. Se eligió un modelo univariado, $UMDA_{cat}$, porque tiene un coste menor que otros modelos probabilísticos más complejos. Además, las bases de datos de patrones se calculan una sola vez y se almacenan en disco para poder reutilizarlas en ejecuciones posteriores. Aunque la eficiencia energética no es el objetivo principal del trabajo, estas decisiones ayudan a evitar cómputo innecesario y hacen que el sistema sea más práctico.

6.2. Objetivos de Desarrollo Sostenible

Aunque este Trabajo Fin de Grado tiene una naturaleza principalmente técnica y experimental, puede relacionarse con algunos Objetivos de Desarrollo Sostenible de la Agenda 2030. Esta relación no debe entenderse como el objetivo principal del proyecto, sino como una forma de analizar su posible impacto desde una perspectiva más amplia.

ODS 9: Industria, innovación e infraestructura

El objetivo con el que el trabajo guarda una relación más directa es el ODS 9, centrado en la industria, la innovación y la infraestructura. El proyecto propone una aplicación de los Algoritmos de Estimación de Distribuciones a un problema combinatorio con restricciones de movimiento. Aunque el caso de estudio es un cubo *fuse*, varias partes del diseño podrían reutilizarse en otros problemas definidos mediante estados, operadores y funciones de evaluación.

En este sentido, el trabajo aporta una pequeña contribución metodológica dentro del ámbito de la optimización y la inteligencia artificial aplicada a problemas discretos. No se trata de una infraestructura industrial, sino de una infraestructura experimental y académica que puede servir como base para desarrollos posteriores.

ODS 4: Educación de calidad

El trabajo también se relaciona con el ODS 4, relativo a la educación de calidad. La memoria describe de forma detallada el proceso de construcción del modelo: la representación del cubo, la definición de los movimientos, el diseño del genoma, la función de evaluación y la estrategia de búsqueda.

Además, el proyecto muestra cómo se puede mejorar progresivamente un algoritmo a partir de pruebas y errores. La evolución de la función de evaluación permite entender por qué una métrica sencilla puede no ser suficiente y por qué es necesario incorporar información más cercana a la estructura real del problema. Por ello, el trabajo puede tener valor formativo para estudiantes interesados en optimización combinatoria, metaheurísticas o resolución automática de rompecabezas.

ODS 12 y ODS 13: uso responsable de recursos computacionales

De forma más indirecta, el trabajo puede relacionarse con el ODS 12, sobre producción y consumo responsables, y con el ODS 13, sobre acción por el clima. La

Impacto del trabajo

ejecución de algoritmos de búsqueda y metaheurísticas implica consumo de recursos computacionales, especialmente cuando se realizan muchas pruebas o se aumentan los parámetros del algoritmo.

En este proyecto se han tomado algunas decisiones para limitar ese coste. Por ejemplo, se utiliza un modelo univariado en lugar de modelos probabilísticos más pesados, se reutilizan las bases de datos de patrones una vez calculadas y se trabaja con presupuestos de evaluación acotados. No se pretende presentar el trabajo como un proyecto medioambiental, pero sí señalar que la eficiencia computacional se ha tenido en cuenta durante el diseño.

El resto de Objetivos de Desarrollo Sostenible no guardan una relación directa con la naturaleza del trabajo. Por este motivo, no se fuerza su inclusión.

Bibliografía

- [1] Richard E. Korf. *Finding Optimal Solutions to Rubik's Cube Using Pattern Databases*. 1997. URL: <https://www.cs.princeton.edu/courses/archive/fall06/cos402/papers/korfrubik.pdf> (visitado 20-10-2025).
- [2] Pedro Larrañaga y Concha Bielza. *Estimation of Distribution Algorithms (EDAs)*. 2024. URL: https://cig.fi.upm.es/wp-content/uploads/EDAs2024_revised.pdf (visitado 20-10-2025).
- [3] Alexander Shmakov Forest Agostinelli Stephen McAleer y Pierre Baldi. *Solving the Rubik's Cube with Deep Reinforcement Learning and Search*. 2019. URL: <https://www.nature.com/articles/s42256-019-0070-z> (visitado 27-10-2025).
- [4] Britannica Editors. *Ernö Rubik and Rubik's Cube*. URL: <https://www.britannica.com/biography/Erno-Rubik> (visitado 03-10-2025).
- [5] Wired Staff. *Jan. 30, 1975: Rubik Applies for Patent on Magic Cube*. 2009. URL: <https://www.wired.com/2009/01/dayintech-0130/> (visitado 03-10-2025).
- [6] The Official Rubik's. *50 Years of Rubik's Cube*. URL: <https://www.rubiks.com/history> (visitado 03-10-2025).
- [7] Wikipedia. *Rubik's Cube*. URL: https://en.wikipedia.org/wiki/Rubik%27s_Cube (visitado 03-10-2025).
- [8] Herbert Kociemba. *The Cubie Level*. URL: <https://kociemba.org/math/cubielevel.htm>.
- [9] Kociemba. *The Facelet Level*. URL: <https://kociemba.org/cube.htm> (visitado 06-11-2025).
- [10] SFU Math Jamie Mullholand. *Representing the cube as a subgroup of S48*. URL: <https://www.sfu.ca/~jtmulhol/math302/puzzles-rc-representation.html> (visitado 06-11-2025).
- [11] William Stallings. *Computer Organization and Architecture*. Prentice Hall, 2006.
- [12] Pelikan Hauschild. *An introduction and survey of estimation of distribution algorithms*. URL: <https://www.sciencedirect.com/science/article/abs/pii/S2210650211000435> (visitado 06-11-2025).
- [13] José Antonio Sierra Ibáñez. *Algoritmos de distribución de probabilidad para el problema de la mochila*. Universidad de Almería, 2018.
- [14] World Cube Association. *WCA Regulations – Article 12: Notation*. URL: <https://www.worldcubeassociation.org/regulations/> (visitado 01-01-2026).
- [15] Herbert Kociemba. *The Coordinate Level*. URL: <https://kociemba.org/math/coordlevel.htm>.
- [16] Vicente P Soloviev, Pedro Larrañaga y Concha Bielza. «EDAspy: An extensible python package for estimation of distribution algorithms». En: *Neurocomputing* (2024), pág. 128043.

- [17] Heinz Mühlenbein y Gerhard Paaß. «From Recombination of Genes to the Estimation of Distributions I. Binary Parameters». En: *Parallel Problem Solving from Nature — PPSN IV*. Vol. 1141. Lecture Notes in Computer Science. Springer, 1996, págs. 178-187.
- [18] Benjamin Doerr y Martin S. Krejca. «The Univariate Marginal Distribution Algorithm Copes Well with Deception and Epistasis». En: *Evolutionary Computation in Combinatorial Optimization (EvoCOP 2020)*. Vol. 12102. Lecture Notes in Computer Science. Springer, 2020, págs. 51-66.
- [19] Nail El-Sourani, Sascha Hauke y Markus Borschbach. «An Evolutionary Approach for Solving the Rubik's Cube Incorporating Exact Methods». En: *Applications of Evolutionary Computation — EvoApplications 2010*. Vol. 6024. Lecture Notes in Computer Science. Springer, 2010, págs. 80-89. DOI: 10.1007/978-3-642-12239-2_9.
- [20] Joseph C. Culberson y Jonathan Schaeffer. «Pattern Databases». En: *Computational Intelligence* 14.3 (1998), págs. 318-334. DOI: 10.1111/0824-7935.00065. URL: <https://doi.org/10.1111/0824-7935.00065>.
- [21] Jessica Fridrich. *My system for solving Rubik's cube*. Accedido: julio de 2026. 1997. URL: <http://www.ws.binghamton.edu/fridrich/system.html>.
- [22] Gilles Roux. *Roux method*. Accedido: julio de 2026. 2003. URL: <http://grrroux.free.fr/method/Intro.html>.
- [23] Herbert Kociemba. *The Two-Phase-Algorithm*. URL: <https://kociemba.org/math/twophase.htm> (visitado 03-07-2026).

Apéndice A

Anexo

«Este capítulo (anexo) es opcional, y se escribirá de acuerdo con las indicaciones del Tutor.»